

FILTERSYNTHESIZER V1.1.0

User Manual

What every control does, which topology to choose, and what to do when the solver says no.

2026-10-04

Screenshots captured at v1.1.0

1. How the app is organized

The sidebar holds the specification. The four tabs turn it into hardware, left to right. Everything recomputes as you edit; there is no Run button.

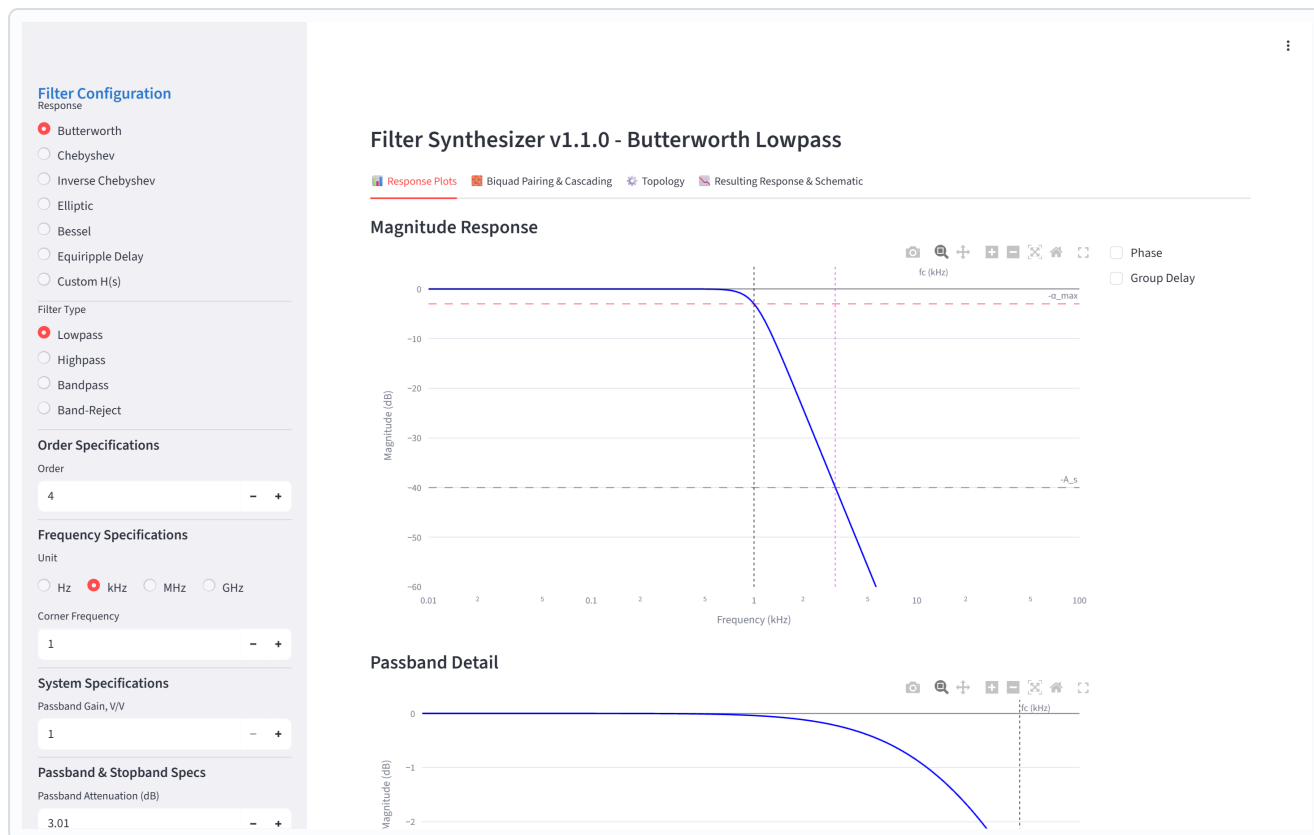


Figure 1. The app as it opens: the specification on the left, the four tabs across the top.

Tab	What it does	What it needs first
Response Plots	Magnitude of the approximation, with phase and group delay on request; group-delay detail for the delay responses; frequency probes; manual notches; at the end, the roots and $H(s)$. For Custom $H(s)$ it starts with the panel where $H(s)$ is typed in	A valid specification
Biquad Pairing & Cascading	Groups the roots into sections and shares out the gain; hands the cascade to the Topology tab	Every zero assigned to a section
Topology	Per section: circuit family, op-amp, component limits; solve , then pick one BOM	The cascade from Tab 2
Resulting Response & Schematic	The cascade built from your picks: Bode plot, Monte-Carlo, LTspice export, PDF report	A picked BOM in every section

In this manual the tabs are numbered in that order: Tab 1 is Response Plots, Tab 4 is Resulting Response & Schematic.

Tabs 1 and 2 are mathematics and update at once, so that is where to iterate on order, corner and gain. Tab 3 is where time is spent: each section is solved on its own, and solving does not choose anything — you pick a row. Tab 4 and the report stay blocked until every section has a pick, and say which sections are still missing.

The colour code. A box with a **blue** border and tint holds controls that change the design: manual notches, the pairing and gain distribution, the convergence and section settings, the Monte-Carlo tolerances, the Custom H(s) panel. An **amber** box is where you pick one of the solver's results — the BOM table in Tab 3. Everything outside these boxes only displays.

What persists and what resets

- **The design is not saved to disk.** It lives in the browser session. Reloading the page (F5) starts over: the sidebar returns to its defaults and every section has to be solved again. The PDF report is the record of a design.
- **The approximation recomputes a moment after each sidebar edit.** A specification you have used before comes back instantly from a cache.
- **Solving runs in the background.** Other sections and tabs stay usable while a section solves. Sections solve in parallel, one processor core each, with one core kept free for the app: on an 8-core machine up to seven sections solve at once, and further ones wait in a queue. The tab header shows how many are solving and how many are queued.
- **The Topology tab refreshes itself every two seconds** while a solve is running — that is how finished solves appear without a click.
- **Results are remembered per configuration.** A section's results belong to everything that produced them: its f_0 , Q and gain, the envelope, the E-series, the op-amp and the convergence settings. Change any of these and the table disappears and the button reads **Solve section** again; change it back and the earlier table returns without a new solve.
- **A pick belongs to the section number.** It survives re-sorting the table and switching tabs. After changing the specification, solve and pick every section again — until you do, a section keeps its previous pick, and Tab 4 and the report use those parts.
- **Switching the Response keeps what you typed.** The Custom H(s) panel, the delay settings and the order survive a switch to another response and back. The op-amp library and imported vendor SPICE models are the only things kept on disk (§7.4).

2. Concepts and vocabulary

Term	Means
Section (stage)	One block of the cascade, realized by one circuit: a complex pole pair (2nd order), a pole pair with an absorbed real pole (3rd order), or a lone real pole (1st order). Zeros assigned to it make it a notch, high-pass or band-pass section. Sections are numbered by rising Q.
Absorption	An odd order leaves a real pole. Absorbing it into a pole-pair section makes that section 3rd order and saves an op-amp. Switched on in Tab 2; off by default.
f_0 , Q, f_p , f_z	A section's pole-pair frequency and quality factor; f_p is its real pole (absorbed or lone); f_z its notch frequency. Shown in each section's header in Tab 3.
K_i	A section's gain constant — the leading coefficient of its transfer function, set by Tab 2's gain distribution. Tab 3 shows the passband gain it implies: $H(0)$ for a low-pass, $H(\infty)$ for a high-pass, $ H(f_0) $ for a band-pass.
Group delay $\tau(f)$	How long the filter delays a signal at frequency f : the negative slope of the phase. A flat τ keeps a pulse's shape. Bessel makes τ maximally flat, Equiripple Delay lets it ripple by $\pm\delta$ to get a sharper magnitude. τ_0 is the nominal delay — $\tau(0)$ for a low-pass.
Lowpass prototype	A normalized low-pass whose passband edge is at $\omega = 1$ rad/s. The app maps it to the chosen filter type and frequencies. Custom $H(s)$ can take one directly, or a complete $H(s)$ that is already the final filter.
Cell	A named circuit that realizes a section, e.g. <code>3LP-gained</code> or <code>2BP-MFB-QE</code> . The name encodes order, response and variant (§7.3).
BOM (candidate)	One complete set of component values for a section. A solve returns many, ranked.
Envelope	The limits a BOM must respect: $C_{\min}...C_{\max}$, $R_{\min}...R_{\max}$, Max R ratio, and the E-series.
Snapping	Resistors are solved as continuous values, then moved to values of your resistor E-series. Capacitors are taken from their E-series from the start and are never snapped.
Sens (sensitivity score)	How strongly the section's transfer-function coefficients move when a component drifts by 1 %. Lower is better. A ranking figure, not a predicted tolerance — Monte-Carlo in Tab 4 is that.
Snap cost	How far the snapped parts leave the section from its target: a weighted sum of the fractional errors in gain, pole frequency, Q and notch depth. 0 is perfect, lower is better. Like Sens, it ranks; it is not a percentage.
Design vs realized	Design is the mathematical target (blue curves). Realized is what the snapped parts do with the chosen op-amp model (red curves).
Sign	Inverting cells contribute -1 to the cascade. An odd number of them inverts the output.

3. Control reference

One table per tab, top to bottom as on screen. The **Key** column is the control's internal name; quote it when reporting a problem. In keys, ***** stands for the section number (or a band or notch index). Controls without a key show "—".

3.1 Sidebar — the specification

The figure shows two panels of a control sidebar. The left panel, titled 'Filter Configuration', has three sections: 'Response' with radio buttons for Butterworth (selected), Chebyshev, Inverse Chebyshev, Elliptic, Bessel, Equiripple Delay, and Custom H(s); 'Filter Type' with radio buttons for Lowpass (selected), Highpass, Bandpass, and Band-Reject; and 'Order Specifications' with a numeric input for 'Order' set to 5. The right panel, titled 'Frequency Specifications', has a 'Unit' section with radio buttons for Hz, kHz (selected), MHz, and GHz; a 'Corner Frequency' section with a numeric input set to 2; and a 'System Specifications' section with a 'Passband Gain, V/V' input set to 1.5.

Figure 2. The sidebar, top and bottom, with the Quick Start example: Butterworth low-pass, order 5, 2 kHz, 1.5 V/V.

Control	Key	Default	What it does
Response	—	Butterworth	The approximation: Butterworth (maximally flat), Chebyshev (passband ripple), Inverse Chebyshev (flat passband, notched stopband), Elliptic (ripple in both, steepest), Bessel (maximally flat group delay), Equiripple Delay (group delay within $\pm\delta$), or Custom H(s) (your own transfer function). Changes the order limits and which boxes appear below. The delay responses and Custom H(s) have sidebars of their own — see the two sections after this table.
Filter Type	<code>widget_filter_type</code>	Lowpass	Lowpass, Highpass, Bandpass, Band-Reject. The band types take two corners and an order per side. For Bessel and Equiripple Delay the radio is <code>widget_filter_type_delay</code> and offers only Lowpass and Bandpass; a caption says why. The choice is kept across the switch: a type the new response does not offer falls back to Lowpass.
Order	<code>widget_sym_order</code>	4	Filter order. For the band types it is the order of each side, labelled Order (LP & HP) . Limits below.

Control	Key	Default	What it does
Asymmetric	<code>is_asym_checkbox</code>	off	Band types only (not for a Chebyshev or Elliptic band-reject): separate orders for the two sides.
LP Order / HP Order	<code>widget_lp_order</code> , <code>widget_hp_order</code>	4 / 4	Shown when Asymmetric is ticked. LP is the upper-edge side, HP the lower-edge side.
Unit	—	kHz	Unit of every frequency box and of the Tab 1 probes.
Corner Frequency	<code>widget_fc</code>	1	Lowpass and Highpass: the passband edge.
Lower / Upper Passband Corner	<code>widget_f1</code> , <code>widget_f2</code>	1 / 2	Band types: the two passband edges.
Passband Gain, V/V	<code>widget_gain</code>	1	Overall passband gain, at least 1. Tab 2 shares it out among the sections.
Passband Attenuation (dB)	—	3.0103	Butterworth, Inverse Chebyshev, Bessel and Equiripple Delay: attenuation at the corner. 0.01–12 dB. For the delay responses it defines where the corner is, so it also moves the poles.
Passband Ripple $\alpha_{\max}$ (dB)	—	1.0	Chebyshev and Elliptic: the passband ripple, in the same box.
Stopband Attenuation A_s (dB)	<code>sym_as_val</code>	40	Minimum stopband attenuation, 10 dB or more. For Inverse Chebyshev and Elliptic it shapes the design; for Butterworth and Chebyshev it only sets the reported stopband edge f_s and the green line on the plot. For Bessel and Equiripple Delay it is the A_s of the stopband order criterion and of the equiripple stopband (§3.2). The key applies to the Inverse Chebyshev and Elliptic band types; elsewhere the box has none.
Lower Stopband A_{sl} / Upper Stopband A_{su} (dB)	<code>as_sl_val</code> , <code>as_su_val</code>	40 / 40	Inverse Chebyshev and Elliptic band types with Asymmetric ticked: one attenuation per stopband.
Response Modifications	—	off	Checkboxes that appear only where they apply — see below.

Order limits

Response	Lowpass / Highpass	Bandpass, per side	Band-Reject, per side	Asymmetric, per side
Butterworth	1–20	1–15	1–14	1–14
Chebyshev	1–20	1–20	1–16	1–10 (Bandpass only)
Inverse Chebyshev	1–20	1–20	1–20	1–10
Elliptic	2–15	2–15	2–15	1–10 (Bandpass only)

Response	Lowpass / Highpass	Bandpass, per side	Band-Reject, per side	Asymmetric, per side
Bessel	1–20 (Lowpass only)	1–10 (prototype order n)	—	—
Equiripple Delay	1–15 (Lowpass only)	1–10 (prototype order n)	—	—
Custom H(s), prototype	1–20	1–15	1–15	—
Custom H(s), complete	at most 30 poles in all			

A value outside the new limits is clamped when you change the response. A delay bandpass is made from a low-pass prototype of order n and has order 2n.

Coupled behaviour worth knowing

- **The two corners push each other apart.** Raising the lower corner to or above the upper one moves the upper corner to twice the lower; lowering the upper corner to or below the lower one moves the lower corner to half the upper.
- **The corner carries over between filter types.** The Lowpass/Highpass corner and the lower band corner are the same stored value.
- **The orders stay in step until you edit HP.** The symmetric order and the LP order are one value; the HP order follows it until you change HP once, after which it keeps its own.
- **Asymmetric stopbands start equal.** Ticking **Asymmetric** copies the common A_s into both A_sl and A_su; unticking copies A_sl back.

Response Modifications

Checkbox	Appears for	Effect
Passband Even Order Modification	Chebyshev and Elliptic, even order, all types except Bandpass	An even-order Chebyshev or Elliptic response normally starts in a ripple valley: the gain at DC (at HF for a high-pass) sits one full ripple below the passband peaks. The modification puts it on a peak instead.
Lower / Upper Passband Even Ord Mod	Asymmetric Chebyshev or Elliptic Band-Reject, per even-order side	The same, for each passband separately.
Stopband Rolloff	Inverse Chebyshev and Elliptic	Moves one pair of transmission zeros to infinity: one notch fewer, and the stopband keeps falling beyond the last notch.
Lower / Upper Stopband Rolloff	Asymmetric Inverse Chebyshev or Elliptic Bandpass	The same, for each stopband separately.
Coincident Stopband Notches	Inverse Chebyshev and Elliptic Band-Reject, total order divisible by 4	The zero pair the rolloff would send to infinity lands at the centre of the stopband instead, so two notches coincide there.

Validation

An invalid specification replaces the whole main area with a red **Mathematical Constraint Violation** box (§6.1). The one you can actually hit: an asymmetric band-reject needs an even total order (LP + HP). Asymmetric Elliptic Bandpass also warns, without blocking, when LP + HP exceeds 15.

Bessel and Equiripple Delay

These two responses are designed for their **group delay**, not their magnitude: Bessel for a delay as flat as possible, Equiripple Delay for a delay that stays within $\pm\delta$ of τ_0 over a wider band, bought with a little ripple. Both are Lowpass or Bandpass only — a high-pass or band-reject passband cannot have a flat delay. They have no Response Modifications.

Filter Configuration

Response

☐ Butterworth
☐ Chebyshev
☐ Inverse Chebyshev
☐ Elliptic
☒ Bessel
☐ Equiripple Delay
☐ Custom H(s)

Filter Type

☒ Lowpass
☐ Bandpass

Bessel: lowpass and bandpass only — a highpass or band-reject passband cannot have a flat group delay.

Order Specifications

Order selection

☐ Manual ☒ From specs

n = 6 — meets the spec

- ✓ Delay error at $f_d \leq 1$ kHz: 0.33 %

Frequency Specifications

Unit

☐ Hz ☒ kHz ☐ MHz ☐ GHz

Specify by

☐ Corner frequency ☒ Group delay

Group delay τ_0 (ms)

0.5

Corner at 3.0103 dB: 0.860517 kHz (n = 6)

System Specifications

Passband Gain, V/V

1.5

Passband & Stopband Specs

Passband Attenuation (dB)

3.01

Stopband Attenuation A_s (dB)

40.00

Delay Specs

Order criterion

☐ Min corner frequency
☒ Flat delay up to f_d
☐ Stopband: A_s at f_s

f_d (kHz)

1

Max delay error ϵ (%)

1.00

Response Modifications

Figure 3. The Bessel sidebar with the order chosen from the specification: the corner follows from $\tau_0 = 0.5$ ms, the order from a delay flat to 1 kHz within 1 %.

Control	Key	Default	What it does
Order selection	<code>widget_delay_order_mode</code>	Manual	Manual: type the order. From specs: the app picks the order from one criterion in Delay Specs below, and shows the result where the Order box was (below the table).

Control	Key	Default	What it does
Order / Prototype order n (BP order = $2n$)	as above	4	Manual mode only. The same stored order as the other responses, clamped to the limits above.
Specify by	<code>widget_delay_anchor</code>	Corner frequency	Lowpass only. Corner frequency : you give f_c , as for any low-pass; τ_0 follows from it. Group delay : you give τ_0 and the corner follows from it and the order.
Group delay τ_0	<code>widget_tau0</code>	1	Shown with Group delay . In the reciprocal of the sidebar unit: s for Hz, ms for kHz, μ s for MHz, ns for GHz. Held exactly: $\tau(0)$ for Bessel, the centre of the $\pm\delta$ band for Equiripple Delay. The derived corner is printed under it: <i>Corner at 3.0103 dB: ... ($n = \dots$).</i>
Corner Frequency / Lower and Upper Passband Corner	as above	—	As for the other responses (above). The corner is where the response is down by the Passband Attenuation .

Delay Specs appears for a bandpass, for Equiripple Delay, and whenever the order comes from the specification (a manual Bessel low-pass needs none of it):

Control	Key	Default	What it does
Bandpass mapping	<code>widget_delay_bp_map</code>	Delay-preserving (arithmetic)	Bandpass only. Delay-preserving moves the low-pass delay shape to the band centre, so the delay stays flat across the band — up to a fractional bandwidth of about 0.3. Classic (geometric) is the usual bandpass transform; the delay is tilted across the band.
Delay ripple $\pm\delta$ (%)	<code>widget_delay_ripple</code>	1.00	Equiripple Delay only: how far τ may deviate from τ_0 in the flat-delay band. 0.05–10 %. Smaller δ approaches Bessel.
Order criterion	<code>widget_crit_*</code>	first option	From specs, Lowpass. One criterion, one number (rows below). The list depends on Specify by : with the corner anchor it starts with Max group delay (latency budget) , with the delay anchor with Min corner frequency ; then Flat delay up to f_d and Stopband: A_s at f_s . Each anchor keeps its own choice (<code>widget_crit_corner</code> , <code>widget_crit_delay</code>). A bandpass always uses the stopband criterion, and says so.
$\tau_0 \leq$	<code>widget_crit_tau_max</code>	0.4	Max group delay: f_c is held, and the app takes the largest order whose delay stays within the budget.
$f_c \geq$	<code>widget_crit_f_min</code>	0.5	Min corner frequency: τ_0 is held, and the app takes the smallest order whose corner reaches this frequency.
f_d	<code>widget_crit_fd</code>	0.5	Flat delay: the smallest order whose delay stays within tolerance up to f_d . The tolerance is $\pm\delta$ for Equiripple Delay.
Max delay error ϵ (%)	<code>widget_crit_eps</code>	1.00	Bessel with the flat-delay criterion: a Bessel delay only sags, so the criterion is $\tau(f) \geq (1 - \epsilon) \cdot \tau_0$ up to f_d . 0.01–50 %.
f_s	<code>widget_crit_fs</code>	3	Stopband: the smallest order with at least A_s (the sidebar's Stopband Attenuation) at f_s . For a bandpass f_s must lie outside the passband; one inside is ignored, with a warning.

The order result. In *From specs* mode a box replaces the Order input: green with **n = ...** and one ✓ line per requirement when the criterion is met, yellow when no allowed order meets it — *no order ≤ ... meets the spec; this is the closest*, the one with the fewest *X* lines and the smallest shortfall, and the design uses it — and blue when no order could be selected — the box says why, and the design keeps the last manual order meanwhile.

A band too wide for the delay-preserving mapping stops the design with a red box naming the B/f_0 limit for that order. Narrow the band, lower the order, or choose **Classic (geometric)**.

Custom H(s)

Custom H(s) takes a transfer function you already have — from a handbook table, MATLAB, a paper — instead of one of the built-in approximations. You type H(s) in a panel at the top of Tab 1 (§3.2); the sidebar holds what it means. Everything after that — pairing, solving, the report — works as for any other design.

There are two modes:

- **Lowpass prototype** (the default): you enter a normalized low-pass whose passband edge is at $\omega = 1$, choose a Filter Type and its frequencies, and the app maps the prototype onto them — as it does internally for the built-in responses.
- **Complete H(s)**: what you enter *is* the filter. The app measures its passband edges and detects its type; there is no Filter Type radio.

Filter Configuration

Response

☐ Butterworth
☐ Chebyshev
☐ Inverse Chebyshev
☐ Elliptic
☐ Bessel
☐ Equiripple Delay
☒ Custom H(s)

Custom Transfer Function

Mode ?

☐ Complete H(s)
☒ Lowpass prototype

Enter H(s) in the panel at the top of Response Plots.

Filter Type

☒ Lowpass
☐ Highpass
☐ Bandpass
☐ Band-Reject

Frequency Specifications

Unit

☐ Hz
☒ kHz
☐ MHz
☐ GHz

Corner Frequency

System Specifications

Gain ?

☐ Normalize
☒ As entered

$G = 1 \text{ V/V}$ (+0.00 dB) from H(s)

Passband & Stopband Specs

Prototype edge attenuation $\alpha = 3.01 \text{ dB}$ (measured at $\omega = 1$)

Stopband reference A_s (dB, plot only) ?

Figure 4. The Custom H(s) sidebar in prototype mode: the mode, the filter type and its corner, the gain mode, and the two reference levels.

Custom Transfer Function

Mode ?

☒ Complete H(s) ☐ Lowpass prototype

Enter H(s) in the panel at the top of Response Plots.

Scale

☒ Normalized (s / ω_n) ☐ Absolute

Filter Type: Lowpass (detected from H(s))

Frequency Specifications

Unit

☐ Hz ☒ kHz ☐ MHz ☐ GHz

Norm. frequency f_n (kHz)

1

Measured at -3.0103 dB; fc = 1 kHz

System Specifications

Gain ?

☐ Normalize ☒ As entered

G = 1 V/V (+0.00 dB) from H(s)

Passband & Stopband Specs

Passband edge level α (dB below peak) ?

3.0103

Stopband reference A_s (dB, plot only) ?

40.00

Custom Transfer Function H(s)

Input form

☒ Coefficients ☐ Factored (f_p, Q) ☐ Factored (Tietze-Schenk) ☐ Roots (d, u)

Power	$\mathcal{F}_p$ Numerator N	$\mathcal{F}_p$ Denominator D
s ⁰	1	1
s ¹	None	2.613126
s ²	None	3.414214
s ³	None	2.613126
s ⁴	None	1

+ -

Row i holds the coefficient of sⁱ (row 0 = the constant term). s is s / ω_n . The gain is implicit: k = b_m / a_n.

Paste coefficients

Numerator N	Denominator D
1	1 2.61313 3.41421 2.61313 1

Order

☒ Descending (MATLAB / numpy) ☐ Ascending (a₀, a₁, ...)

Apply

Reset to the example (Butterworth n = 4)

4 poles (0 real) - 0 finite zeros (0 at the origin, 0 j ω pairs)

Edges at -3.01 dB: 1 kHz - Entered peak gain G = 1 V/V (+0.00 dB) at 0.0703731 kHz - Detected: Lowpass - Coefficient conditioning: 1.5e-05 dB

$|H(j\omega_n)| = -3.01$ dB re the peak; f_p dB / f_n = 1. Handbooks disagree on what $\omega = 1$ means (-3 dB, ripple edge, unit delay ...). f_n must be the table's reference frequency, not the desired corner.

Figure 5. Complete mode: the type is detected from H(s), and the Scale decides how the entered numbers are read.

Top to bottom:

Control	Key	Default	What it does
Mode	<code>widget_custom_mode</code>	Lowpass prototype	Complete H(s) or Lowpass prototype , as above.
Scale	<code>widget_custom_scale</code>	Normalized (s / ω_n)	Complete mode only. Normalized : the entered s is s / ω_n , with $\omega_n = 2\pi \cdot f_n$ from the box below — the way handbook tables are printed. Absolute : s in rad/s, as typed. With the Tietze–Schenk form the radio is greyed (<code>widget_custom_scale_ts</code>): those tables are always normalized.
Filter Type	as above	Lowpass	Prototype mode: the target type, as for the other responses. Complete mode: no radio, but the line <i>Filter Type: ... (detected from H(s))</i> . A response that is none of the four types stops the design.
Unit	—	kHz	As above.
Norm. frequency f_n	<code>widget_custom_fn</code>	1	Complete mode, Normalized scale: the frequency the table's $\omega = 1$ stands for. It must be the table's own reference frequency, not the corner you want: handbooks disagree on what $\omega = 1$ means (-3 dB, ripple edge, unit delay ...). Under it, the measured passband edges.

Control	Key	Default	What it does
Corner Frequency / Lower and Upper Passband Corner	as above	—	Prototype mode: where the prototype's $\omega = 1$ lands.
Passband definition	<code>widget_custom_pbdef</code>	Corners	Prototype mode, Bandpass and Band-Reject. Corners: the lower and upper passband edges. Normalized width: Centre frequency f_0 (the same box as f_n above) and Δ , as in handbook frequency-transformation examples; the derived corners are printed below.
Normalized width $\Delta = (f_2 - f_1) / f_0$	<code>widget_custom_bw</code>	0.5	With Normalized width . The centre is geometric.
Gain	<code>widget_custom_gain_mode</code>	As entered	As entered: the peak gain G of your $H(s)$ is the passband gain, and the sidebar prints $G = \dots V/V$ from $H(s)$. G may be below 1. Normalize: $H(s)$ is scaled so its peak equals the Passband Gain box that then appears; K or A_0 in the panel is greyed and recalculated.
Passband edge level α (dB below peak)	<code>widget_custom_alpha</code>	3.0103	Complete mode. The passband edges are the outermost frequencies where $ H $ is α below its peak; the type is detected against the same level. 0.01–40 dB. In prototype mode the box is replaced by the prototype's measured edge attenuation, $ H(j1) $ below its peak.
Stopband reference A_s (dB, plot only)	<code>widget_custom_as</code>	40	Only draws the $-A_s$ line on the plots: the entered $H(s)$ fixes the stopband.

Custom $H(s)$ has no Order box and no Response Modifications — both are in the entered $H(s)$. In prototype mode a bandpass or band-reject mapping does not keep a flat group delay, so a Bessel prototype loses it; the panel says so.

3.2 Tab 1 — Response Plots

The tab shows the approximation you specified, top to bottom: the magnitude plot, a passband detail, the group-delay detail (delay responses only), the stopband-edge readout, the frequency probes, the manual notches and, at the end, the roots and the transfer function. For **Custom $H(s)$** the tab starts with the panel where $H(s)$ is entered (last part of this section).

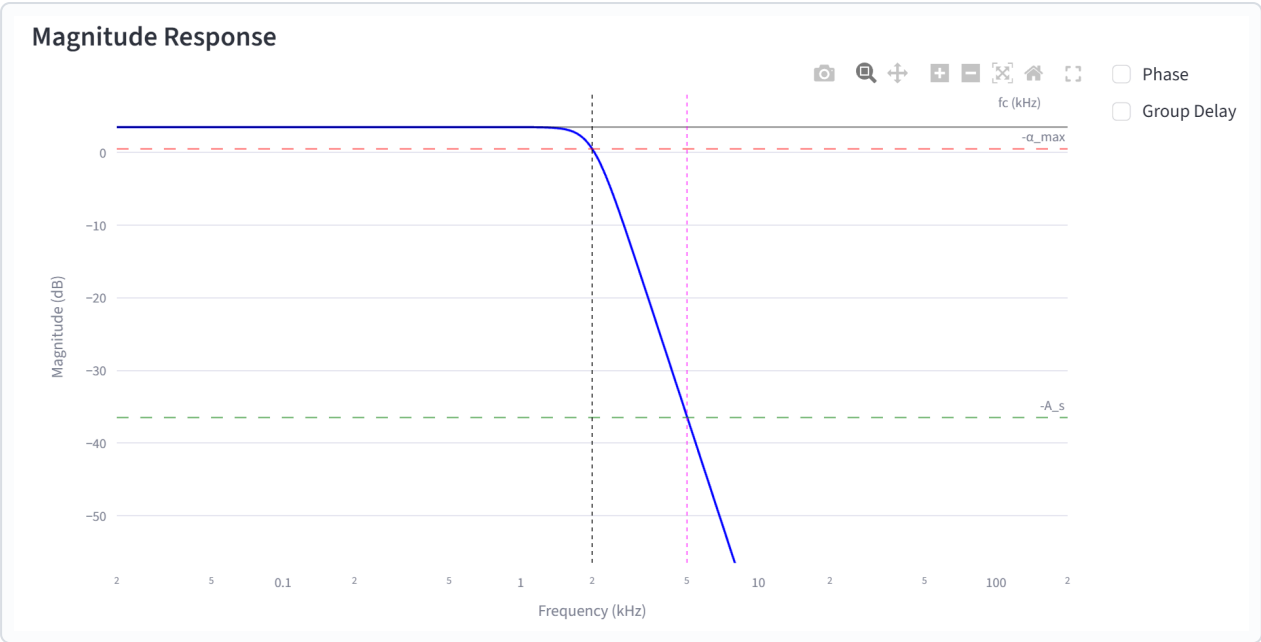


Figure 6. Magnitude response of the Quick Start example: flat at +3.52 dB (1.5 V/V), down to the red $-\alpha_{\max}$ line at the 2 kHz corner.

The **Magnitude Response** plot marks $-\alpha_{\max}$ (red dashed), $-A_s$ (green dashed), the corner (black dashed) and the calculated stopband edge f_s (magenta dotted). **Passband Detail** below zooms into the passband — for a band-reject, **Lower Passband** and **Upper Passband** side by side. Under the plots, the **Calculated Stopband Edge (f_s)** box is green when the stopband meets A_s , yellow when a pinned notch pushed the edge out, and red when the stopband never reaches A_s . Custom $H(s)$ has no such box: its stopband is whatever was entered.

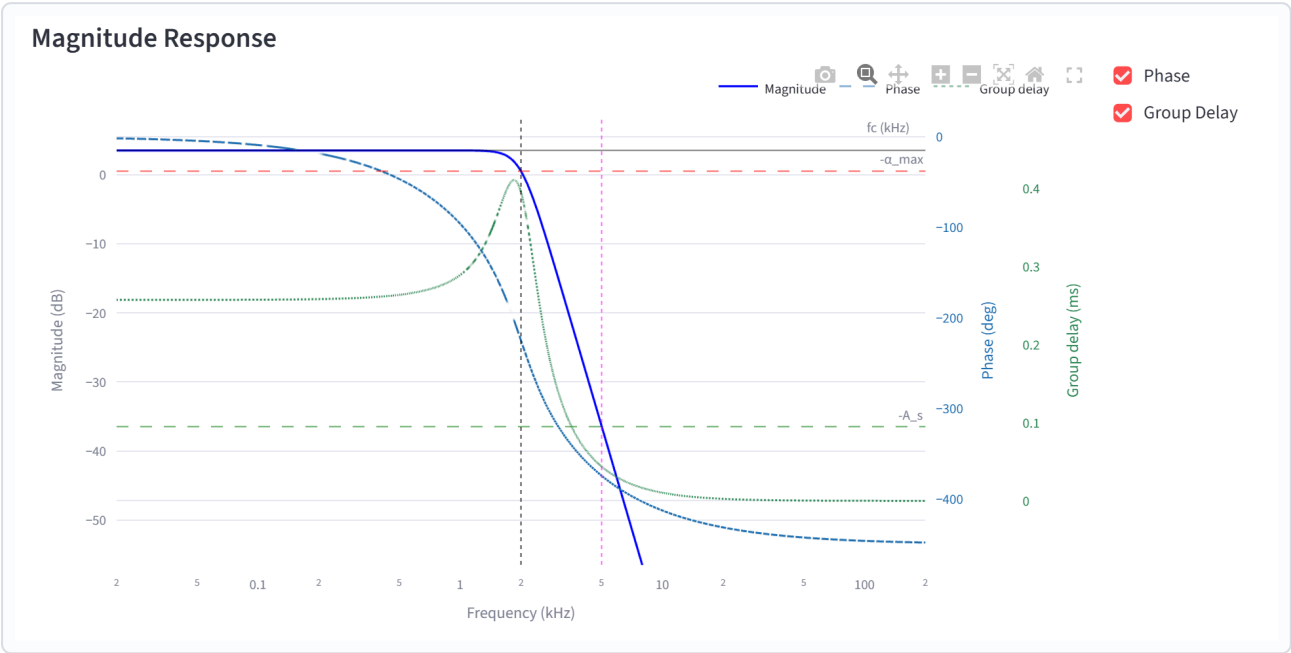


Figure 7. Phase and group delay overlaid on the magnitude plot, each on its own right-hand axis.

Control	Key	Default	What it does
Phase	—	off	Overlays the phase (dashed, degrees) on the magnitude plot, on a right-hand axis.

Control	Key	Default	What it does
Group Delay	—	off	Overlays the group delay (dotted, ms) on its own right-hand axis — next to the phase axis when both are on. The axis is scaled to the bulk of the curve, so the spikes at notch frequencies run off the plot instead of flattening it.
Probe 1 / 2 / 3	probe_*_*	corner, 2× and 10× corner (Lowpass)	Gain and phase of the design at the typed frequency, printed beside each box. Each filter type keeps its own probes; the second * is the filter type.
Pin <i>N</i>	pin_notch_*	off	Manual notches, one row per zero pair the order allows. Butterworth, Chebyshev, Bessel and Equiripple Delay (Manual Notch Placement): an unpinned row reads ∞ ; pinning adds a finite notch at the typed frequency. Inverse Chebyshev and Elliptic (Manual Notch Tuning): unpinned rows show the computed notch; pinning freezes it so you can move it.
Notch frequency	val_notch_*	2× corner (Lowpass)	The pinned notch's frequency, in the sidebar unit.
Lower / Upper Stopband Notches	pin_notch_hp_* , val_notch_hp_* , pin_notch_lp_* , val_notch_lp_*	off	Bandpass: the same, one column per stopband.
Equiripple Magnitude Stopband	widget_ems	off	Bessel and Equiripple Delay low-pass, with a manual order and the corner anchor (otherwise a one-line note says what it needs). See below.
Active <i>N</i>	ems_active_*	the first [(<i>n</i> – 1)/2] rows, at least one	With the equiripple stopband on, replaces Pin : how many notches the app places. Each solved frequency is shown read-only.

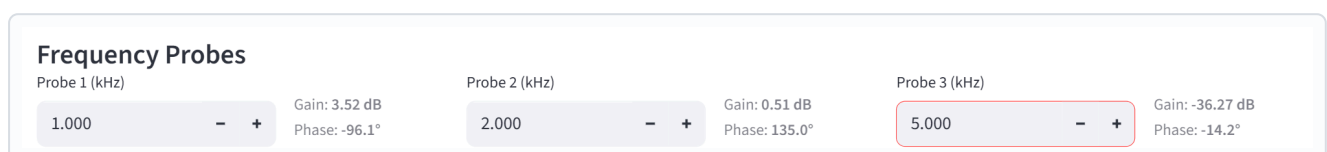


Figure 8. The frequency probes: the design's gain and phase at three frequencies of your choice.

Manual notches are not offered for Elliptic band filters, for Elliptic Lowpass/Highpass above order 8, for a delay bandpass, or for Custom $H(s)$ (its zeros are entered in the panel); the tab says so.

Notches on a delay response sit in the stopband only: a pinned notch must lie above the corner, and one in the passband is ignored. Notches add no group delay. With the corner anchor the poles are scaled to hold the corner, so τ_0 shrinks slightly; with the τ_0 anchor the corner moves instead.

Equiripple magnitude stopband

A Bessel or Equiripple Delay low-pass has a soft stopband. With **Equiripple Magnitude Stopband** ticked, the app places the active notches so that every stopband hump sits exactly at $-A_s$ — an Inverse-Chebyshev-like stopband on a flat-delay filter. You choose only how many notches: each one costs 40

dB/dec of far-stopband roll-off. The poles are scaled to keep the corner where it was, so the delay keeps its shape and τ_0 drops by the scale factor. Your pins are parked while the mode is on and come back when you switch it off.

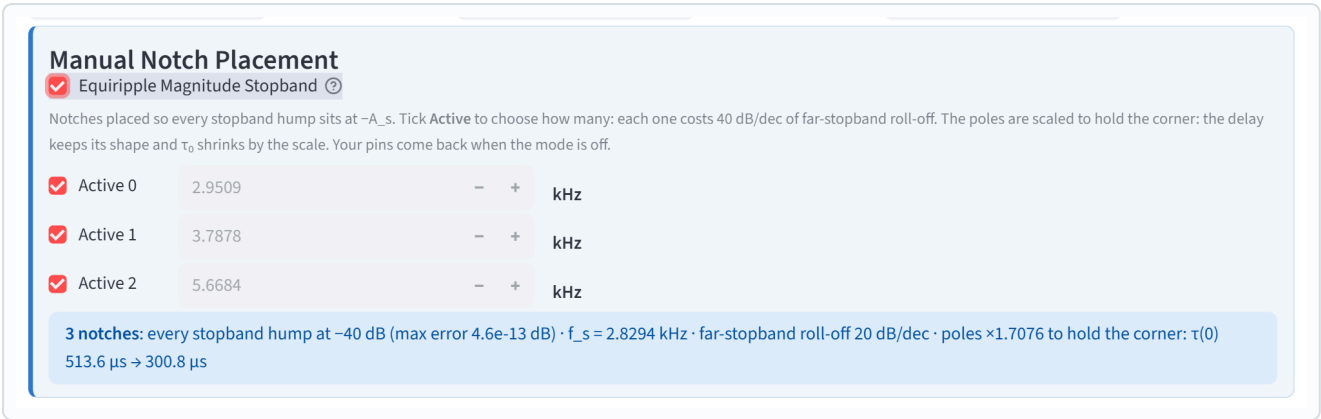


Figure 9. Equiripple stopband on a 7th-order Equiripple Delay low-pass: the notches are solved, and the readout states what they cost.

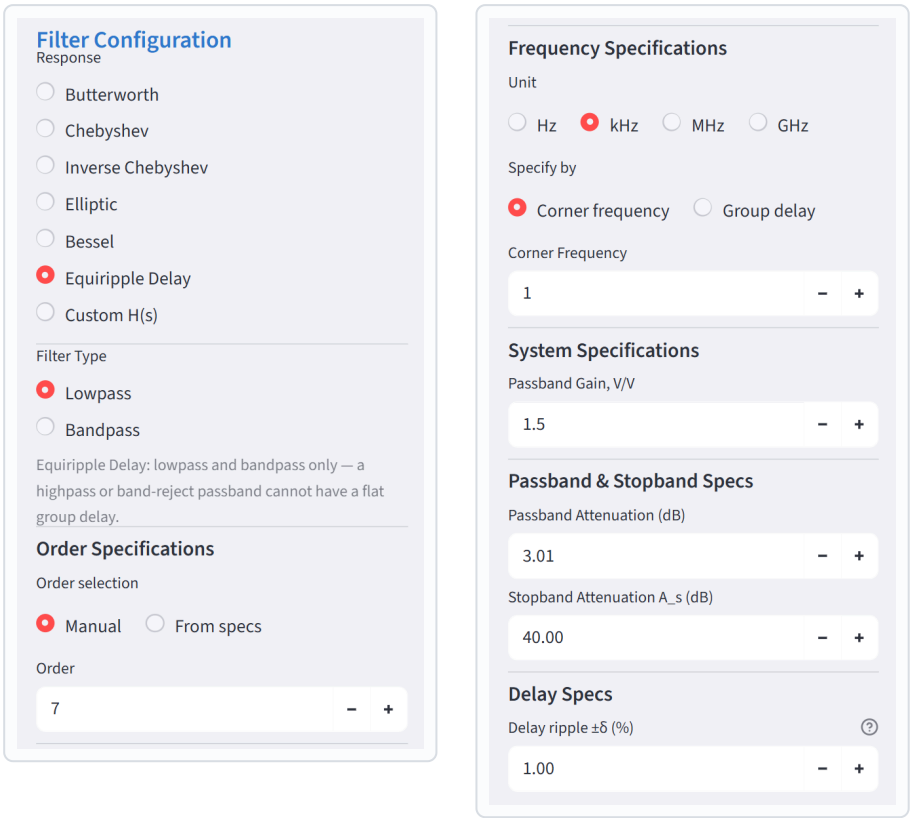


Figure 10. The specification of that example: Equiripple Delay low-pass, manual order 7, corner anchor.

The readout under the rows gives the number of notches, the hump level and the largest error, f_s , the far-stopband roll-off (or a flat floor at $-A_s$ when every pole pair has a notch), and the pole scale with $\tau(0)$ before $\rightarrow$ after. It turns yellow if the notch solve did not converge; with no row active it says the filter is the plain delay low-pass.

Group Delay Detail

Shown for Bessel and Equiripple Delay. A row of figures, then a zoomed $\tau(f)$ plot with the tolerance band and the flat-delay edge:

Figure	Low-pass	Bandpass
Delay	Group delay $\tau(0)$; Equiripple Delay adds Nominal delay	Delay at centre and Centre
Corner	Corner (... dB)	—
Flat band	Flat delay ($\pm\delta$ or $-\epsilon$) to — where τ leaves the tolerance	Flat delay ($\pm...$) — the band, and Delay p-p $f_1...f_2$, the peak-to-peak delay variation across the passband
Max section Q	the highest Q of any pole pair — a first hint of how hard the hardware will be	same



Figure 11. Group Delay Detail for the Bessel example: $\tau(0)$, the corner, the flat-delay edge and the highest section Q, above the zoomed delay curve.

A caption reports the pole scale when notches moved the poles. The plot marks f_d (magenta dotted) when the flat-delay criterion is in use.

Roots & Transfer Function

The grey frame at the end of the tab. **Domain Scale** at its top drives everything inside: the map, the root tables, K and $H(s)$.

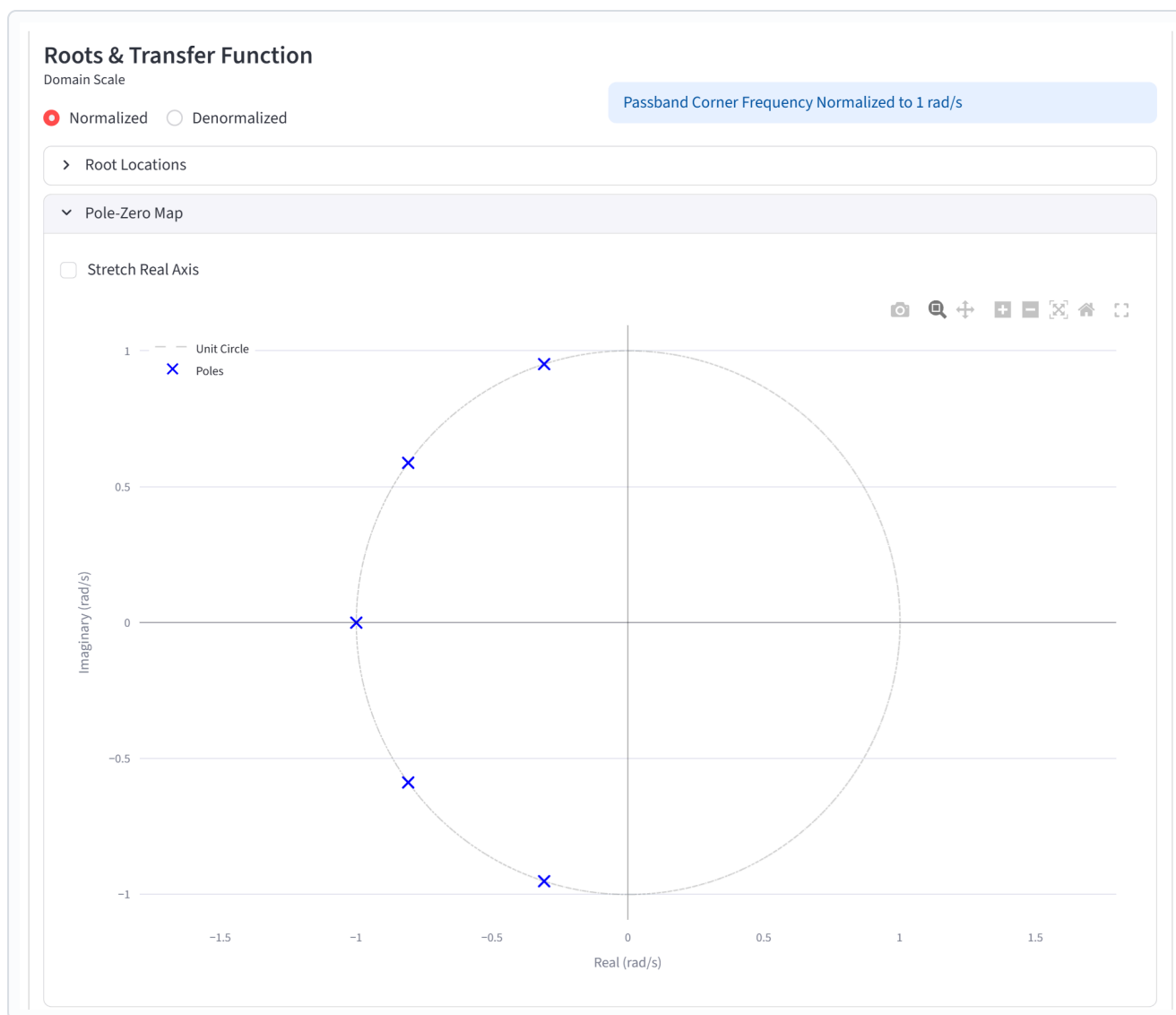


Figure 12. The pole-zero map, normalized: five poles on the unit circle — one real, two complex pairs.

Control	Key	Default	What it does
Domain Scale	<code>scale_roots</code>	Normalized	Normalized: corner (band centre for band types) at 1 rad/s, unity gain. For Custom $H(s)$, normalized to $\omega_n = 2\pi \cdot f_n$, so the tables show what you typed. Denormalized: real frequencies, and K includes the passband gain.
Pole-Zero Map Units	<code>unit_roots</code>	rad/s	Denormalized only: rad/s or Hertz for the map, the tables and $H(s)$.
Stretch Real Axis	—	off	Inside Pole-Zero Map . Stretches the real axis — useful when poles crowd the imaginary axis.
Magnification	—	1.0	Shown when stretching: 0.25–10.
Form	<code>tf_form_roots</code>	Expanded (Isolated Gain)	Inside Transfer Function $H(s)$: Expanded (Isolated Gain) , Expanded (Distributed Gain) or Factored (Cascaded Biquads) .

Three collapsed expanders follow the scale:

- **Root Locations** — the poles and zeros, and the **System Gain Constant (K)** with its units.
- **Pole-Zero Map** — the map, with the stretch control.

- **Transfer Function H(s)** — H(s) in the form the radio picks. The rendered equation wraps to fit the page; the grey box under it is the same expression as one LaTeX line, ready to copy, followed by the coefficient table.

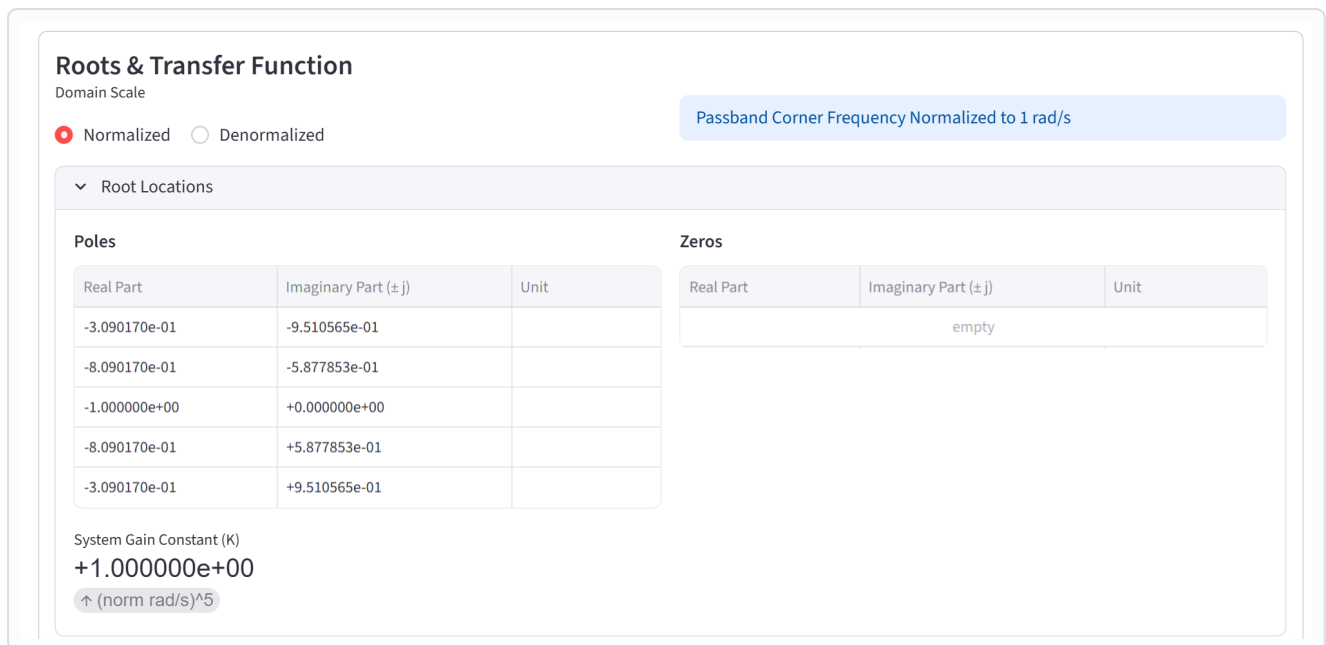


Figure 13. Root Locations: five poles, one real and two complex pairs, and the gain constant K.

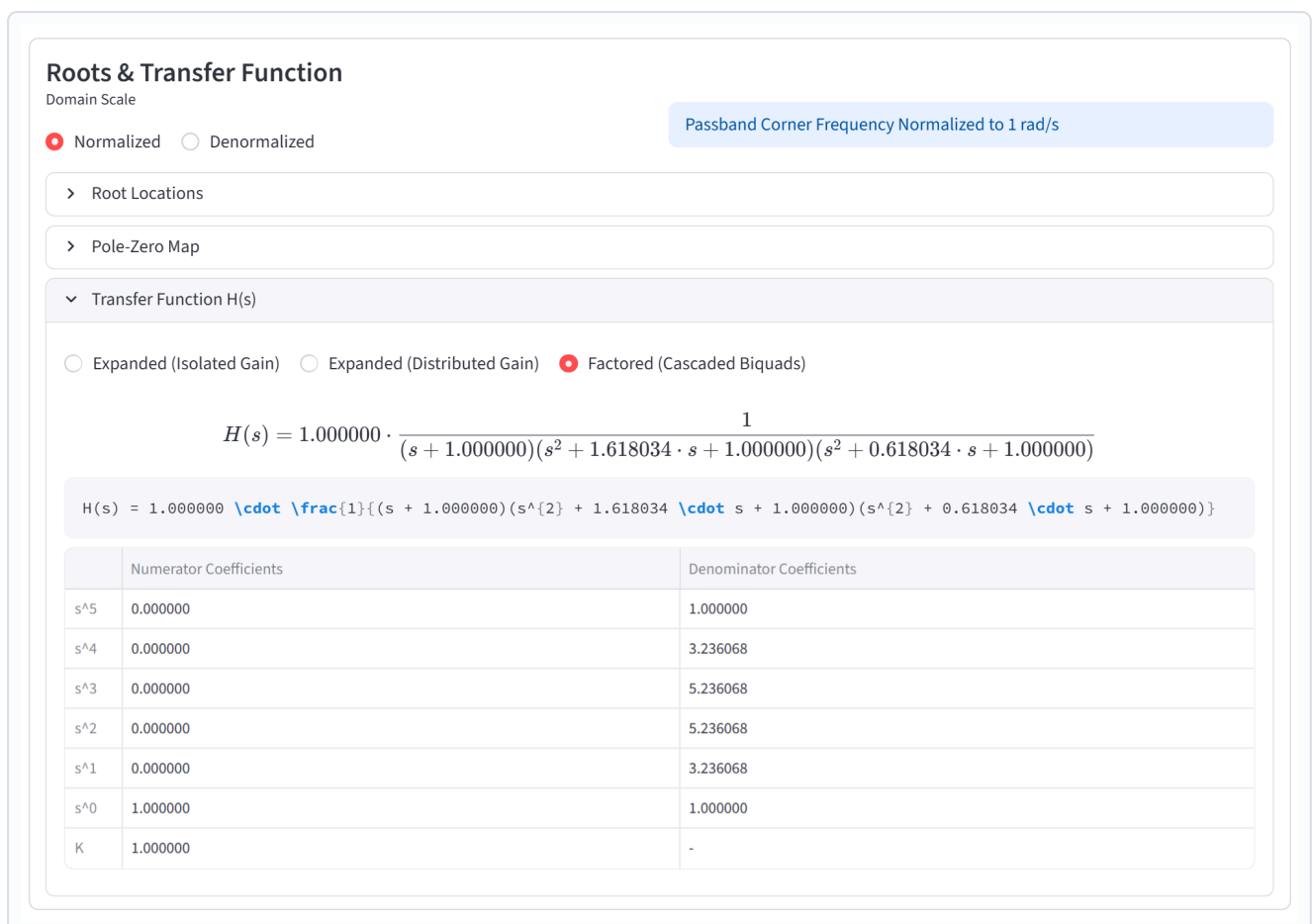


Figure 14. The whole frame with H(s) in factored form: one quadratic per pole pair.

The Custom H(s) panel

With Response **Custom H(s)**, the tab starts with a blue panel, **Custom Transfer Function H(s)**, where the transfer function is typed in. Pick the form your source uses; each form keeps its own entries, so switching does not convert one into another. Errors stop the design under the panel, so the input can be fixed in place.

Custom Transfer Function H(s)
Input form

☐ Coefficients ☒ Factored (f_0 , Q) ☐ Factored (Tietze–Schenk) ☐ Roots (σ , ω)

Normalized: $\omega = 1$ is the prototype's passband edge.

Pole pairs — $s^2 + (\omega_0/Q) \cdot s + \omega_0^2$

$\Rightarrow f_0$ (normalized)	$\Rightarrow Q$
1	0.541196
1	1.306563

+ -

Zero pairs — Q_z blank = $j\omega$ pair (notch)

$\Rightarrow f_z$ (normalized)	$\Rightarrow Q_z$ (blank = $j\omega$)
empty	

+ -

Real poles — at $-\omega$

$\Rightarrow f$ (normalized)
empty

+ -

Real zeros — σ_z , + = RHP

$\Rightarrow \sigma_z$ (normalized)
empty

+ -

K (constant before the monic factors)

1

Reset to the example (Butterworth $n = 4$)

4 poles (0 real) · 0 finite zeros (0 at the origin, 0 $j\omega$ pairs)

Target edges: 1 kHz · prototype edge $\alpha = 3.01$ dB · Entered peak gain $G = 1$ V/V (+0.00 dB) at DC · Detected: **Lowpass**

$|H(j\omega_n)| = -3.01$ dB re the peak; $f_{-3\text{dB}} / f_n = 1$. Handbooks disagree on what $\omega = 1$ means (−3 dB, ripple edge, unit delay...): f_n must be the table's reference frequency, not the desired corner.

Figure 15. The panel in its default form, Factored (f_0 , Q), holding the built-in example: a 4th-order Butterworth.

Control	Key	Default	What it does
Input form	<code>custom_form</code>	Factored (f_0 , Q)	Coefficients, Factored (f_0, Q), Factored (Tietze–Schenk) or Roots (σ, ω) — below.
The tables	<code>custom_tbl_*_*</code>	Butterworth $n = 4$	One editable table per block of the form. Rows are fixed; type into the cells. Typed text is kept as typed.
+ / −	<code>custom_add_*</code> , <code>custom_del_*</code>	—	Under each table: add a row, remove the last row.
Zeros at the origin n_0	<code>custom_n0</code>	0	Factored (f_0 , Q), complete mode only (a low-pass prototype has none).
K (constant before the monic factors)	<code>custom_K_f0q</code>	1	Factored (f_0 , Q).
A_0 (DC gain)	<code>custom_A0</code>	1	Tietze–Schenk.
K (leading constant)	<code>custom_K_roots</code>	1	Roots (σ , ω). Labelled C (leading constant of $1/H$, $K = 1/C$) in C form.

Control	Key	Default	What it does
Leading constant	<code>custom_kform_roots</code>	K — gain function $H(s)$	Roots (σ, ω): K for $H(s) = K \cdot \prod (s - z_i) / \prod (s - p_i)$, or C — attenuation function $1/H(s)$ (Saal) for tables that give $1/H(s) = C \cdot \prod (s - p_i) / \prod (s - z_i)$; then $K = 1/C$. Switching reinterprets the typed number; it does not convert it.
Paste coefficients: Numerator N / Denominator D	<code>custom_paste_num</code> , <code>custom_paste_den</code>	empty	Coefficients form: paste a whole polynomial: numbers separated by spaces, commas, semicolons or new lines; brackets are ignored, so MATLAB's <code>[1 2.613 ...]</code> pastes as is.
Order	<code>custom_paste_order</code>	Descending (MATLAB / numpy)	Order of the pasted coefficients; Ascending ($a_0, a_1, \dots$) for tables that start with the constant term.
Apply	<code>custom_paste_apply</code>	—	Writes the pasted polynomials into the table. An empty box leaves its column unchanged.
Paste roots: Poles / Zeros	<code>custom_paste_poles</code> , <code>custom_paste_zeros</code>	empty	Roots form: one root per line — <code>-0.5+0.866j</code> , MATLAB's <code>-0.5000 + 0.8660i</code> , or <code>re im .A</code> full list with both conjugates (the output of <code>roots(den)</code>) is de-duplicated.
Apply	<code>custom_paste_apply_roots</code>	—	Writes the pasted roots into the tables.
Reset to the example (Butterworth $n = 4$)	<code>custom_reset</code>	—	Restores the example in every form; keeps the form, scale and <code>f_n</code> .

The four forms:

- **Coefficients** — numerator N and denominator D side by side; row i holds the coefficient of s^i (row 0 is the constant term). The gain is implicit: $k = b_m / a_n$.
- **Factored (f_0, Q)** — pole pairs $s^2 + (\omega_0/Q) \cdot s + \omega_0^2$ as f_0 and Q ; real poles at $-\omega$ as f ; zero pairs as f_z and Q_z (Q_z blank = a pair on the $j\omega$ axis, a notch); real zeros as σ_z ($+$ = right half-plane).
- **Factored (Tietze–Schenk)** — the all-pole tables of Tietze–Schenk and TI's *Op Amps for Everyone*: $H(s_n) = A_0 / \prod (1 + a_i s_n + b_i s_n^2)$, one row per stage, $b = 0$ for a real pole. Always normalized. For zeros, use the (f_0, Q) form.
- **Roots (σ, ω)** — poles and zeros as real and imaginary parts; $\omega > 0$ stands for a conjugate pair.

Custom Transfer Function H(s)

Input form

☐ Coefficients
 ☐ Factored (f_0, Q)
 ☐ Factored (Tietze–Schenk)
 ☒ Roots (σ, ω)

Normalized: $\omega = 1$ is the prototype's passband edge.

Poles (σ, ω) — $\omega > 0 =$ conjugate pair (normalized)

σ (Re)	ω (Im ≥ 0)
-0.382683	0.92388
-0.92388	0.382683

+ -

Zeros (σ, ω) — $\omega > 0 =$ conjugate pair (normalized)

σ (Re)	ω (Im ≥ 0)
empty	

+ -

Leading constant ?

☒ K — gain function H(s)
 ☐ C — attenuation function 1/H(s) (Saal)

K (leading constant)

1

▼ Paste roots

One root per line ($-0.5+0.866j$, MATLAB $-0.5000 + 0.8660i$, or 're im'). Full lists with both conjugates (roots(den) output) are de-duplicated.

Poles

Zeros

Apply

Reset to the example (Butterworth n = 4)

4 poles (0 real) · 0 finite zeros (0 at the origin, 0 $j\omega$ pairs)

Target edges: 1 kHz · prototype edge $\alpha = 3.01$ dB · Entered peak gain G = 0.999999 V/V (-0.00 dB) at 0.0390148 kHz · Detected: **Lowpass**

$|H(j\omega_n)| = -3.01$ dB re the peak; $f_{-3dB} / f_n = 1$. Handbooks disagree on what $\omega = 1$ means (-3 dB, ripple edge, unit delay...): f_n must be the table's reference frequency, not the desired corner.

Figure 16. The Roots form: poles and zeros as (σ, ω) , the leading-constant choice, and the paste box.

Frequencies are normalized ($\omega = 1$ is the prototype's passband edge) in prototype mode, multiples of f_n in complete mode with the Normalized scale, and in the sidebar unit (rad/s for roots and coefficients) with Absolute.

K and A_0 under Normalize. With the sidebar's Gain set to **Normalize**, the constant field is greyed and shows the value that puts the peak of $H(s)$ at the Passband Gain; the K / C radio is greyed too. Switch Gain to **As entered** to type it.

Diagnostics under the inputs:

- the counts — poles (real ones), finite zeros (at the origin, $j\omega$ pairs), and repeated roots merged;
- the passband edges at $-\alpha$ (complete mode) or the target edges and the prototype's own edge attenuation (prototype mode);
- the **entered peak gain G** and where it occurs, the **detected** type, and the coefficient conditioning;
- for a normalized entry, $|H(j\omega_n)|$ relative to the peak and f_{-3dB} / f_n — the quickest check that f_n is the table's reference frequency;
- warnings, and a **Pairing pre-flight** list when the automatic pairing would give a section a numerator no circuit realizes.

What the panel accepts. The poles must be stable — none at the origin, on the $j\omega$ axis or in the right half-plane — and $H(s)$ proper (no more finite zeros than poles). Zeros must lie on the $j\omega$ axis or at the origin: a real zero, or a zero pair off the axis, is refused, because no cell realizes it yet. A complete $H(s)$ whose response is not a low-pass, high-pass, bandpass or band-reject is refused too. A negative K is used as $|K|$: the sign is set by the circuits. Each refusal names the root or row at fault (§6.1).

Elsewhere in the app, a Custom $H(s)$ design differs only in small ways: the Pairing tab hides **Equalize DC and HF gains** for an asymmetric band-reject, the Topology tab's overall target gain is the entered G , and the report lists the entry instead of the approximation's parameters (§8).

3.3 Tab 2 — Biquad Pairing & Cascading

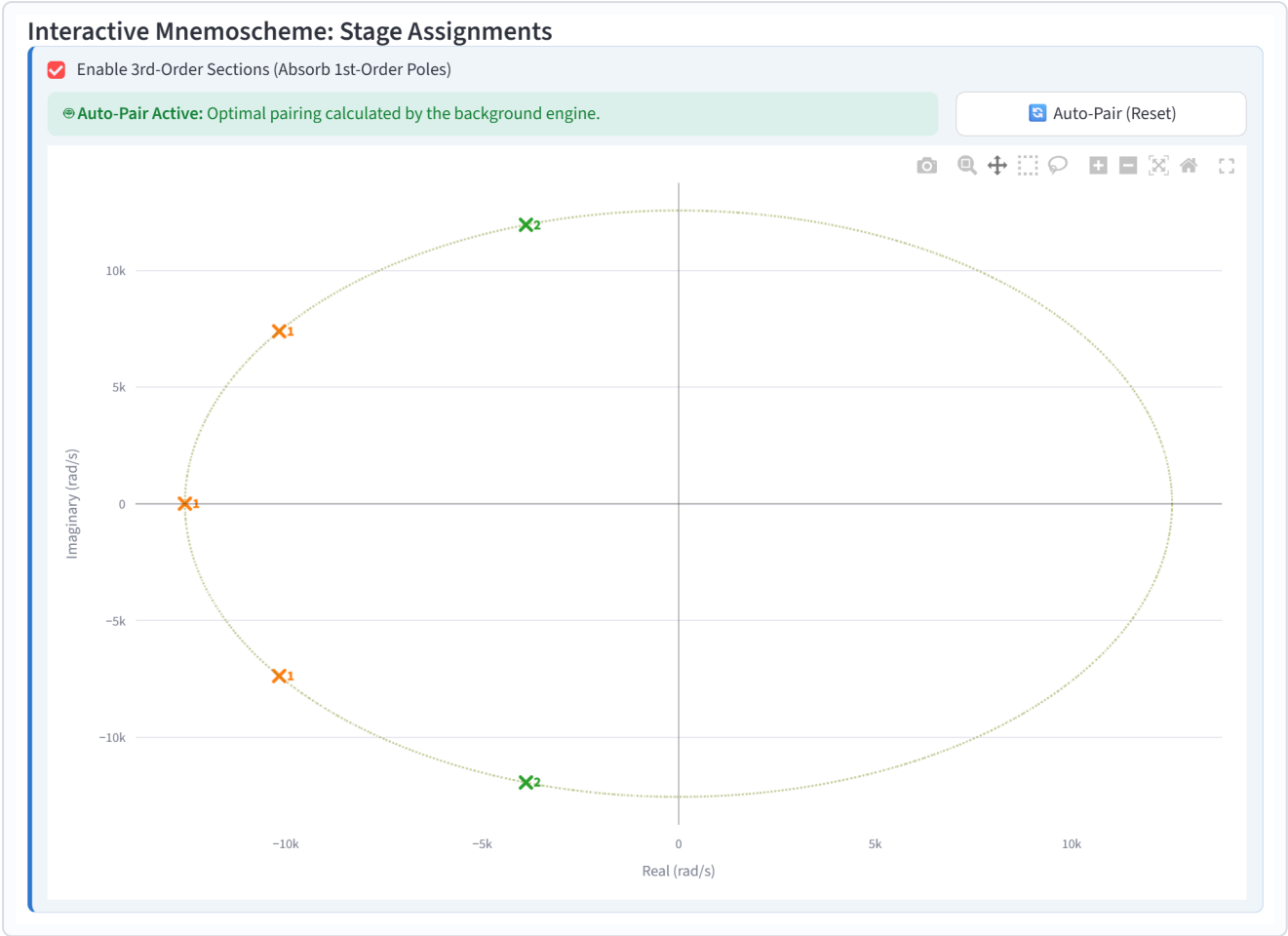


Figure 17. Tab 2 with absorption on: the real pole joins the low-Q pair as section 1 (orange); the high-Q pair is section 2 (green).

Control	Key	Default	What it does
Hardware Target Units	<code>unit_pair</code>	rad/s	Units of the mnemoscheme and of the section tables below. The Topology tab always works in Hz.
Enable 3rd-Order Sections (Absorb 1st-Order Poles)	—	off	Shown only when the filter has a real pole. Ticked: each real pole is absorbed into a pole-pair section, which becomes 3rd order. Unticked: the real pole is a 1st-order section of its own.
The mnemoscheme	<code>pz_mnemo_chart</code>	automatic pairing	The pole-zero map with each section in its own colour. Clicking edits the pairing (below).

Control	Key	Default	What it does
 Auto-Pair (Reset)	—	—	Discards manual edits and restores the automatic pairing.
Remaining Gain Distribution	—	Distribute Remaining Gain Evenly	The pairing first sets every section to unity passband gain; the Calculated Remainder is the factor still needed to reach the specified passband gain (the gain over unity). This decides where it goes: shared evenly, all to the first stage, or all to the last stage. Band-Reject adds Equalize DC and HF gains of LP and HP sections , which makes both passbands land on the specified gain.

Editing the pairing by hand. Click a pole to select it, then click a zero to move that zero into the pole's section, or a real pole to absorb it (needs the checkbox), or the same pole again to cancel — or, on an absorbed real pole, to detach it into its own section. The banner turns to **Manual Override Active**; sections are renumbered by rising Q after every edit. Until every zero belongs to a section, the gain distribution and the section tables are withheld and the cascade is not handed on.

Each section below the map shows its transfer function (rendered and as a LaTeX line) and a table: K_i , ω_0 or f_0 , Q, the notch ω_z or f_z , the real pole ω_p where there is one, and **Peak Mag (V/V)** — the section's own maximum gain.

3.4 Tab 3 — Topology & Hardware Synthesis

Each section has a header line — order, fp for a real pole, f_0 , Q, and f_z for a notch — then a blue box with its settings expander, its gain control, the cell caption and the **Solve section** button. The caption names the cell the gain selects, e.g. *Cell: 3LP-gained · $H(0) = 1.5$ (from K) · tol ±2%*: a section gain within ±2 % of 1 uses the unity cell, above it the gained cell, below it an attenuating cell. Below the box, once solved, the amber box with the candidates table.

At the top of the tab, above **Convergence Settings**, the **Batch mode** switch decides whether each section has its own op-amp and component envelope (off, the default) or all sections share one (on).

Batch mode

Control	Key	Default	What it does
Batch mode — shared envelope & op-amp	hw_batch	off	On: one op-amp and one component envelope — C and R range, Max R ratio, E-series — for every section, set once in  Shared settings — all sections (Batch mode) , and a Solve all sections button. Each section keeps its own topology family, family options and gain, and you still pick a row in each section.
Solve all sections	hw_solve_all	—	Batch mode only. Queues every section that has no result for its current settings; they solve in parallel, one core each (\$1). A section that already has a result keeps it — use its own Re-solve to repeat it.

Topology & Hardware Synthesis

☒ Batch mode — shared envelope & op-amp ?

Shared settings — all sections (Batch mode)

Op-amp model

Ideal (no op-amp limits) ▾

Component envelope

C_min (μF)			R_min (kΩ)			Max R ratio	
6.80e-5	-	+	0.3000	-	+	500.00	- +
C_max (μF)			R_max (kΩ)				
1.00e-2	-	+	2000.0	-	+		

Capacitor E-series (single)

☐ E3 ☐ E6 ☒ E12

Resistor E-series (multiple)

☐ E12 ☐ E24 ☒ E48 ☐ E96

> Convergence Settings

Solve all sections 0 / 2 sections with a BOM
 ○ S1: not solved · ○ S2: not solved

Figure 18. Batch mode: one shared op-amp and envelope above all sections, and Solve all sections.

Beside the button a status line counts the sections that have a BOM, how many are solving or queued and how many parallel workers there are, then one entry per section:  with its BOM count,  solving,  queued,  not solved yet,  no BOM, not solvable, or an error.

> Convergence Settings

Solve all sections 0 / 2 sections with a BOM · 2 solving · 31 parallel workers
 S1: solving ·  S2: solving

Section 1 · order 3 · $f_p = 2,000\text{ Hz}$ · $f_s = 2,000\text{ Hz}$ · $Q = 0.618$

> Section 1 — topology settings

Cell: 3LP-gained · $H(0) = 1.22$ (from K) · tol ±2% ☐ Custom DC gain Solve section

 Solving... (other sections and tabs stay responsive)

Section 2 · order 2 · $f_p = 2,000\text{ Hz}$ · $Q = 1.618$

> Section 2 — topology settings

Cell: 3LP-gained · $H(0) = 1.22$ (from K) · tol ±2% ☐ Custom DC gain Solve section

 Solving... (other sections and tabs stay responsive)

Figure 19. Solve all sections in progress: both sections solve at the same time.

Switching is lossless. The first time Batch mode is switched on, the shared settings start from the first section's values. Each section's own values are kept while Batch mode is on and come back when it is switched off; the shared values are kept too. Because results belong to their settings (§1), switching may show **Solve section** again on sections whose own settings differ from the shared ones — switching back brings their tables back.

With Batch mode on, each section's expander reads **Section N — topology settings** and holds only the family and its options; a caption points to the shared box.

Convergence Settings

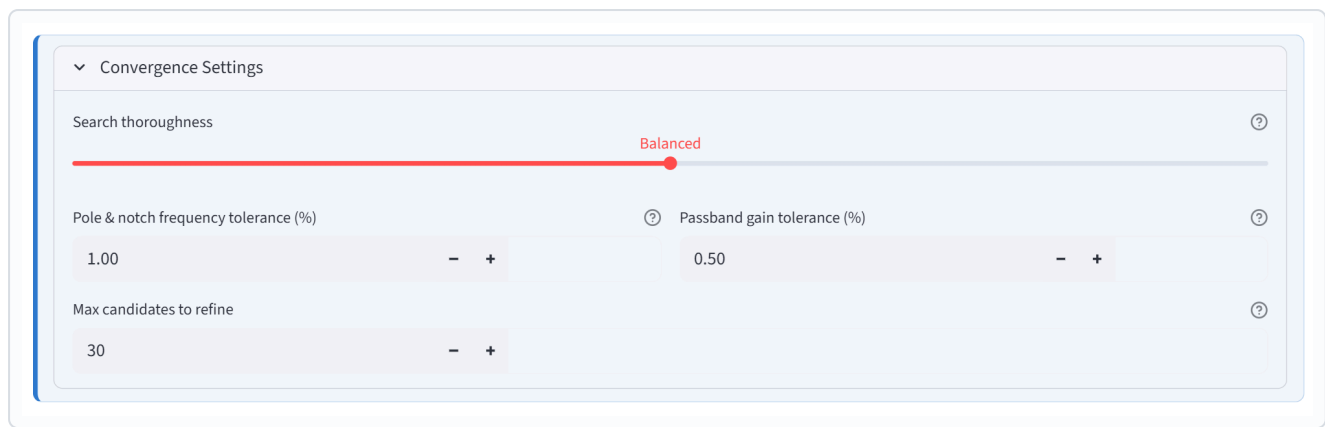


Figure 20. Convergence Settings. The defaults are right for a first run.

These apply to every section.

Control	Key	Default	What it does
Search thoroughness	<code>hw_effort</code>	Balanced	Fast / Balanced / Thorough: how many starting points the search tries and how many minima it keeps. Thorough finds more candidates and takes longer.
Pole & notch frequency tolerance (%)	<code>hw_pole_tol_pct</code>	1.00	How far a candidate's realized f_0 — and a notch section's f_z — may sit from the target; used by the notch search path. Q is not gated by it.
Passband gain tolerance (%)	<code>hw_gain_tol_pct</code>	0.50	How far a candidate's realized passband gain may sit from the target: DC gain for low-pass and notch cells, HF gain for high-pass.
Max candidates to refine	<code>hw_topk</code>	30	Only the best N ideal solutions, by sensitivity, are corrected for the op-amp and snapped — the expensive step for gained and notch cells. Lower is much faster and lists fewer BOMs. 1–500.

Leave these alone until a solve returns nothing *and* the envelope is already wide; then raise **Search thoroughness**. Lower **Max candidates to refine** when a gained or notch section is slow. Loosening the tolerances admits candidates that miss the target by more — a last resort.

Per-section settings

Section 1 · order 3 · $f_p = 2,000 \text{ Hz}$ · $f_0 = 2,000 \text{ Hz}$ · $Q = 0.618$

Section 1 — topology & component settings

Topology family

☒ VCVS (Sallen-Key) ☐ MFB (Friend) ☐ AM (Ackerberg-Mossberg)

Op-amp model

Ideal (no op-amp limits)

Component envelope

C_min (μF)

6.80e-5

-

+

R_min (kΩ)

0.3000

-

+

Max R ratio

500.00

-

+

C_max (μF)

1.00e-2

-

+

R_max (kΩ)

2000.0

-

+

Capacitor E-series (single)

☐ E3 ☐ E6 ☒ E12

Resistor E-series (multiple)

☐ E12 ☐ E24 ☒ E48 ☐ E96

Cell: 3LP-gained · $H(0) = 1.5$ (from K) · tol ±2%

☐ Custom DC gain

Solve section

Figure 21. Section settings: family, op-amp, component envelope and E-series, all at their defaults.

⚙️ **Section N — topology & component settings** (Batch mode off). In Batch mode the op-amp, envelope and E-series rows move to the shared box, with the same keys ending in the shared tag instead of the section number.

Control	Key	Default	What it does
Topology family	hw_fam_*	VCVS (Sallen-Key); MFB for a 3rd-order band-pass	VCVS (Sallen-Key), MFB (Friend) or AM (Ackerberg–Mossberg). §4 helps choose. A 3rd-order band-pass section has cells only in MFB, so it starts on MFB; a later choice of yours stays.
Op-amp model	hw_opamp_choice_*	Ideal (no op-amp limits)	Ideal , a library part, or Custom... — see <i>The op-amp library</i> below. With a real part the solver corrects for it and checks the BOM against its non-ideal model (§5). Under the picker a caption gives the part's A _{ol} , GBWP, Ro and origin (built-in, built-in · edited, or user part).
A _{ol} (V/V), GBWP (Hz), Ro (Ω)	hw_aol_*, hw_gbwp_*, hw_ro_*	1e5, 1e6, 1200	Custom... only: open-loop gain, gain-bandwidth product, output resistance.
C _{min} , C _{max} (μF)	hw_cmin_*, hw_cmax_*	6.80e-05, 1.00e-02	Capacitor window: 68 pF to 10 nF.
R _{min} , R _{max} (kΩ)	hw_rmin_*, hw_rmax_*	0.3000, 2000.0	Resistor window: 300 Ω to 2 MΩ.

Control	Key	Default	What it does
Max R ratio	hw_ratio_*	500	Largest over smallest resistor in one BOM. A wider spread is rejected. An MFB band-pass alone needs about $4\text{-}Q^2$, so 500 allows Q up to about 11.
Capacitor E-series	hw_cser_*	E12	E3, E6 or E12 — one. Capacitor values come from this series directly. E3 is not usable yet (§6.3).
Resistor E-series	hw_rser_*_*	E48	E12, E24, E48, E96 — any combination. Resistors are snapped to the union. With none ticked, E48 is used.
Equalize R, C values	hw_ameq_*	on	AM only. The handbook balanced design: $R_5 = R_6 = R_7 = R_8$ and $C_2 = C_3$. Untick to free R_5/R_6 and C_2/C_3 for a closer E-series fit; $R_7 = R_8$ stays matched.
Gained MFB	hw_mfbgain_*	off	MFB, 2nd-order high-pass-notch sections: solve the unity/gained realization instead of the attenuating one. Its HF gain lives in a narrow band, so Custom HF gain is switched on and pre-filled with the band's middle.
Lower Sensitivity MFB	hw_mfbls_*	off	MFB, low-pass-notch sections: solve the low-sensitivity branch (Q set passively, $S_Q < 1$, a deeper null with a real op-amp).
Eliminate R1	hw_mfbbs_nor1_*	off	2nd-order low-pass notch with Lower Sensitivity MFB ticked: drop R1 for a 7-part cell. The DC gain is then pinned to a narrow window, so Custom DC gain is switched on and pre-filled.

The op-amp library

The **Op-amp model** list is a library: the parts shipped with the app (§5) plus your own, kept in a file of your own that survives upgrades (§7.4). You can save a set of **Custom...** values as a named part, and adjust any part — for your supply voltage or load, say — without touching the shipped values.

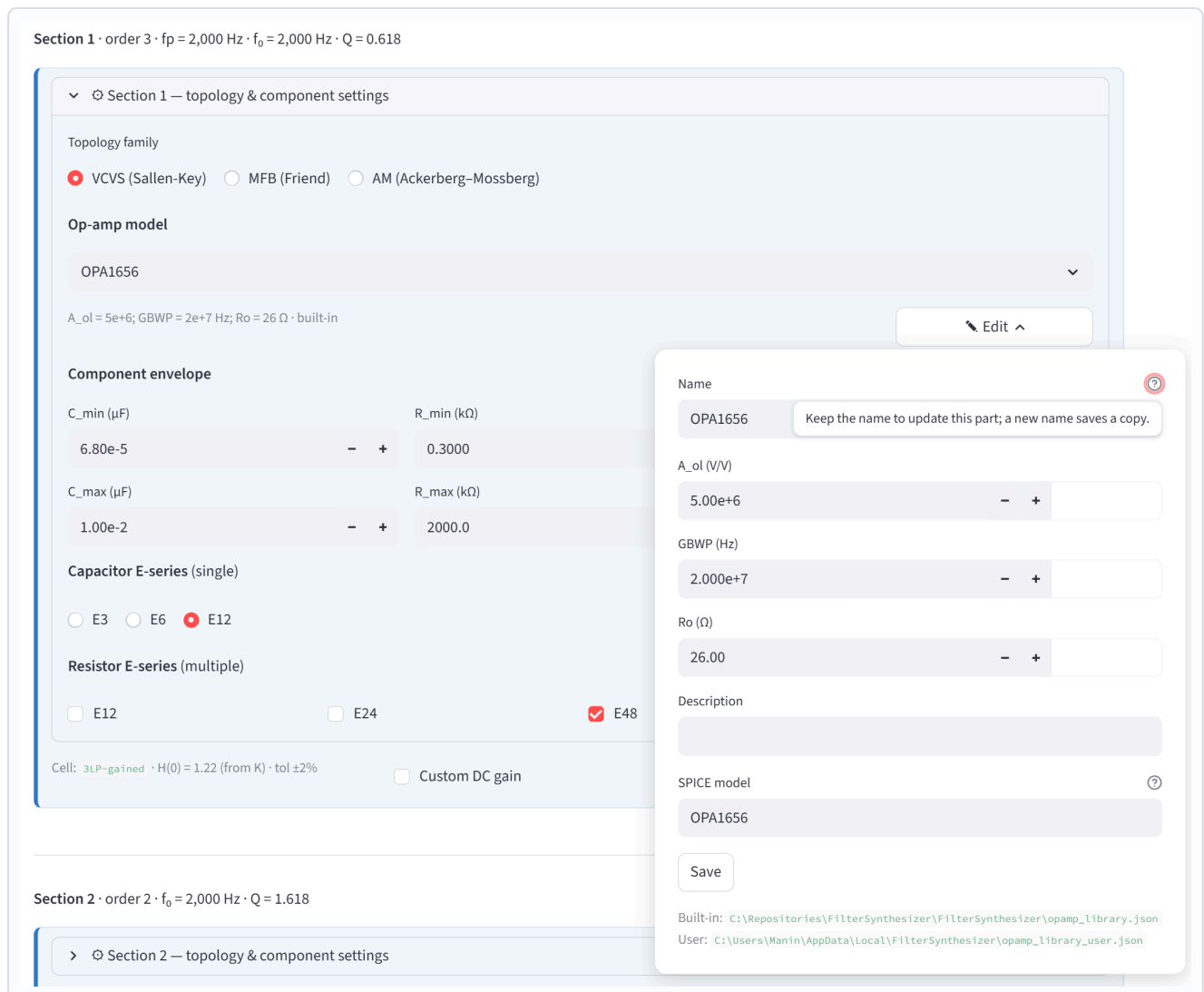


Figure 22. The Edit popover of a library part: its parameters, its LTspice model, and where the two library files are.

Control	Key	What it does
Edit	—	Beside the caption of a library part. Opens a popover with the part's fields.
Name	<code>hw_oped_name_*_*</code>	Keep it to update this part; type a new name to save a copy under it.
A_{ol} (V/V), GBWP (Hz), R_o (Ω), Description	<code>hw_oped_aol_*_*</code> , <code>hw_oped_gbw_*_*</code> , <code>hw_oped_ro_*_*</code> , <code>hw_oped_desc_*_*</code>	The part's values.
SPICE model	<code>hw_oped_spice_*_*</code>	The LTspice model the export uses for this part (§9): the name of an op-amp in the app's LTspice library, e.g. TL072H. Empty = the generic model built from A_{ol} , GBWP and R_o .
Save	<code>hw_oped_save_*</code>	Writes the part to your user library. Editing a built-in part stores an override; the shipped values are never changed. The section switches to the saved part.
Revert to shipped values	<code>hw_oped_revert_*</code>	Only for an edited built-in part: removes your override.

Control	Key	What it does
Delete part	hw_oped_del_*	Only for a part of your own: removes it; the section falls back to Ideal.
Save as part	hw_opnew_name_*	Custom... only: a name for the current A _{ol} , GBWP and Ro. It must not match an existing part (case does not count).
Save to library	hw_opnew_save_*	Saves that part; the section switches to it.

Figure 23. Custom... values, ready to be saved as a named part.

A green or red note under the picker confirms each save, revert or delete for ten seconds. The popover's last line shows the paths of the built-in and the user library files. A library change is seen by every section's list at once. If a library file cannot be read, a yellow warning above the picker says why and the app carries on with what it could read.

A part that was in an older library and is gone now — TL072, LM358 and NE5532 were replaced by TL072H, LM358B and TLV9001-4 in 1.1.0 — falls back to Ideal.

1st-order sections

A real pole that is not absorbed becomes a 1st-order section. Its expander is shorter, **⚙ Section N — component settings**: the op-amp, **C_{max}** (the pole is realized at the three E-series capacitor values nearest below it), and the two E-series. The resistor range is fixed at 100 Ω–10 MΩ. It needs no family and no Solve button: it is solved in closed form as soon as it appears. In Batch mode it takes the op-amp, C_{max} and E-series from the shared box.

Near-notch sections

A section whose transmission zero lies close to f_0 — but not on it, as an Inverse Chebyshev or Elliptic band design can produce — is solved as a low-pass-notch or high-pass-notch section (LPn / HPn), whose cells place the zero where it is. Treating it as a pure notch would move the zero onto f_0 and change the response.

When the zero is within 5 % of f_0 , which realization fits better cannot be told from the mathematics alone, so the section is solved **twice**: on its LPn/HPn cells and on the family's pure-notch (2N) cells. A caption says so:

i Near-notch section: f_z is 1.4 % from f_0 (forcing it onto f_0 changes the response by up to $3.13\times$ the section gain). Solved on both 2HPn-AM, 2HPn-AM-C1s and the pure-notch 2N-AM, 2N-AM-C1s ; the BOMs are ranked together by snap cost.

That example is section 4 of an Elliptic bandpass — order 9 per side, 1.9–2.2 kHz, $\alpha_{\max}$ 1 dB, A_s 40 dB — with $f_0 = 1910$ Hz, $Q = 114$ and $f_z = 1884$ Hz.

Both sets of BOMs are scored against the true target, zero included, and the table is sorted by **Snap cost** by default: a pure-notch row that cannot place the zero carries a high snap cost and sinks. One **Solve section** runs both solves.

Section 4 — topology & component settings

Topology family

☐ VCVS (Sallen-Key)

☐ MFB (Friend)

☒ AM (Ackerberg-Mossberg)

AM (Ackerberg-Mossberg) is the classic THREE-op-amp state-variable biquad: one Miller integrator closed through an actively-compensated integrator and a unity inverter. It covers every 2nd/3rd-order section kind (LP / LP-notch, HP / HP-notch, band-pass, pure-notch). Every AM cell is inverting and the passband gain is a free component ratio. The matched inverter pair $R7 = R8$ gives active GB compensation (Q error $\propto (f_0/f_t)^2$ rather than $Q \cdot f_0/f_t$) — the reason to spend three op-amps; see `AM_NONIDEAL_ANALYSIS.md`. Low-pass and band-pass each offer two taps (out1 and the classic out2 realization), solved together and ranked.

Op-amp model

Ideal (no op-amp limits)

Component envelope

C_min (μF)

6.80e-5

-

+

R_min (kΩ)

0.3000

-

+

Max R ratio

500.00

-

+

C_max (μF)

1.00e-2

-

+

R_max (kΩ)

2000.0

-

+

Capacitor E-series (single)

☐ E3

☐ E6

☒ E12

Resistor E-series (multiple)

☐ E12

☐ E24

☒ E48

☐ E96

☒ Equalize R, C values

Cell: 2HPn-AM · $H(\infty) = 0.84$ (from K) · tol ±2%

☐ Custom HF gain

Re-solve

i Near-notch section: f_z is 1.4 % from f_0 (forcing it onto f_0 changes the response by up to 3.13× the section gain). Solved on both 2HPn-AM, 2HPn-AM-C1s and the pure-notch 2N-AM, 2N-AM-C1s; the BOMs are ranked together by snap cost.

Sort by

Snap cost

Descending

	#	Topology	Sens	Snap cost	HF gain	C1	C2	C3	R1	R2	R4	R5	R6	R7	R8
☐	0	2HPn-AM-C1s	1.40	5.33	0.830	100p,8.2n	10n	10n	--	10.5k	909k	8.25k	8.25k	8.25k	8.25k
☐	1	2N-AM-C1s	1.40	7.19	0.786	100p,8.2n	10n	10n	--	10.5k	909k	8.25k	8.25k	8.25k	8.25k
☐	2	2HPn-AM-C1s	1.40	8.46	0.832	120p,8.2n	10n	10n	--	10.5k	909k	8.25k	8.25k	8.66k	8.66k
☐	3	2N-AM-C1s	1.40	10.59	0.786	120p,8.2n	10n	10n	--	10.5k	909k	8.25k	8.25k	8.66k	8.66k
☐	4	2HPn-AM	1.40	11.41	0.820	8.2n	10n	10n	--	10.5k	953k	8.25k	8.25k	8.66k	8.66k
☐	5	2N-AM	1.40	11.43	0.786	8.2n	10n	10n	--	10.5k	953k	8.25k	8.25k	8.66k	8.66k
☐	6	2HPn-AM-C1s	1.40	13.12	0.835	150p,8.2n	10n	10n	--	10.5k	909k	8.25k	8.25k	8.25k	8.25k

Figure 25. The near-notch section on AM: the caption, and one table sorted by snap cost in which the high-pass-notch rows, which place the zero where it is, rank above the pure-notch rows.

A near-notch needs a component spread of about $1/|(f_z/f_0)^2 - 1|$ on the VCVS and MFB cells. When such a section comes back empty, the diagnosis says so and suggests AM, which sets f_z and f_0 with independent ratios (§6.4).

User Manual · v1.1.0

33

Solving and the candidates table

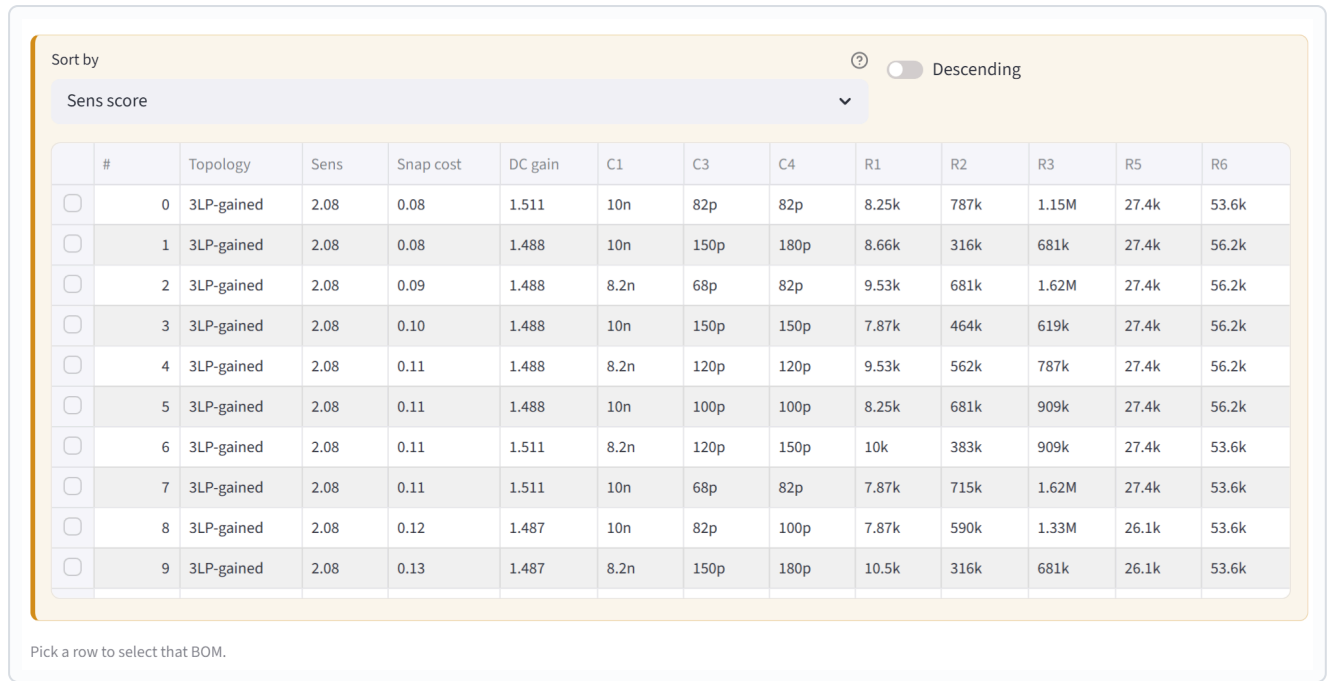


Figure 26. Candidate BOMs for section 1, ranked by sensitivity. Component columns adapt to the cells that were solved.

Control	Key	Default	What it does
Solve section / Re-solve	hw_solve_*	—	Starts the solve in the background. Greyed while it runs, and when the chosen family has no cell for this section. Reads Re-solve when results for these exact settings exist.
Sort by	hw_sortf_*	Sens score (Snap cost for a near-notch section)	Sens score, Snap cost, the gain column, or any component column. Sorts by numeric value, so 8.25k comes before 78.7k.
Descending	hw_sortd_*	off	Reverses the order.
The table	hw_df_*	no row	Tick the box at the left of a row to pick that BOM. Rows are striped for reading.
Select solution #	hw_pick_*	0	Only on Streamlit versions without row selection: type the row's #.

Reading a row. # is the row's place after sorting; **Topology** the cell; **Sens** and **Snap cost** as in §2; the gain column (**DC gain**, **HF gain**, **DC/HF gain** or **Center gain**) is the passband gain the snapped parts actually give, op-amp included. Then one column per component the solved cells use. By default rows are ranked by Sens rounded to two decimals, with ties broken by the lower snap cost. A capacitor shown as 1n, 2n is two capacitors in parallel.

After a pick, a green box repeats the choice (*Selected #0: 3LP-gained · snap_cost 0.08 · sens 2.08 · DC gain 1.511 V/V*), the BOM is listed — **Capacitors** as chosen, resistors snapped with the pre-snap ideal value in grey — and the schematic is drawn with designators prefixed by the section number (1R1, 1C1 ...) and the op-amp's name under U. With a real op-amp, a caption confirms *BOM checked against exact non-ideal op-amp physics*.

Control	Key	What it does
↓ SVG	*_svg_*	The annotated schematic as a vector file. Here and in Tab 4.

Control	Key	What it does
↓ PNG	*_png_*	The same as a bitmap. Only present when PNG export works (§6.8).

Overall filter

At the foot of the tab, once every section has a pick: **Overall realized DC gain** (HF gain for a high-pass) — the product of the sections' realized gains, signed — and **Overall realized passband gain**, which scales that by the design's passband-to-DC ratio. A band-pass shows the centre gain, measured on the realized cascade at the passband peak; a band-reject shows **LF passband gain** and **HF passband gain** separately. For Custom H(s) the target is the entered peak gain G. A yellow box warns when the output is inverted (§6.5).

3.5 Tab 4 — Resulting Response & Schematic

The cascade rebuilt from the BOMs you picked, top to bottom: the HF warning (when there is one, §5), **Monte-Carlo tolerances**, the **Bode — design vs realized** plot, the **Section schematics**, the **LTspice export** and **Generate Report**. Until every section has a pick the tab shows which sections are missing, and the report button is greyed (§6.5).

Monte-Carlo tolerances

The blue box perturbs every resistor and capacitor of the picked BOMs; op-amp parameters stay fixed.

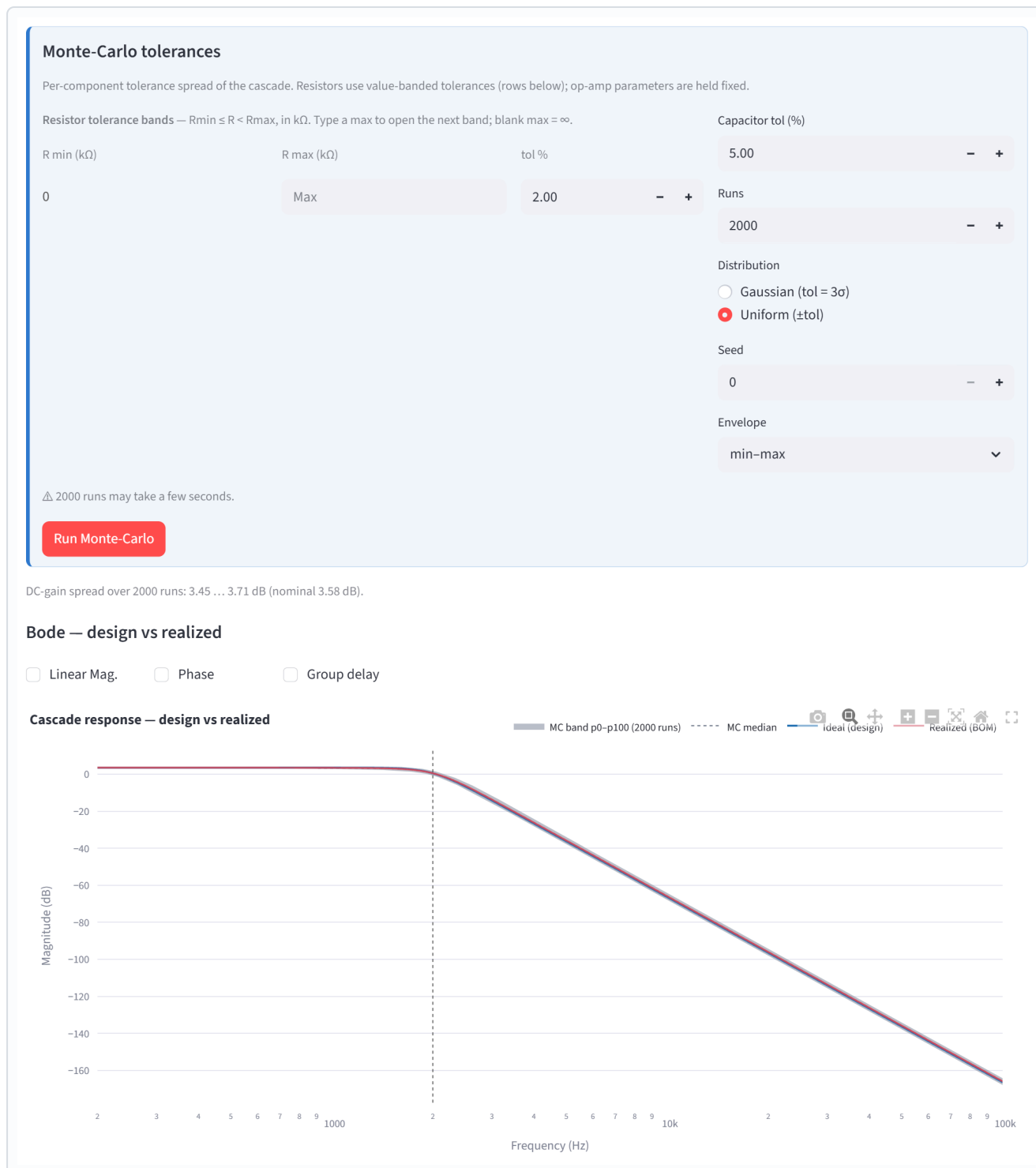


Figure 27. Monte-Carlo with 2 % resistors and 5 % capacitors: the grey band covers every run, and the caption gives the DC-gain spread.

Control	Key	Default	What it does
R max (k Ω)	<code>rtol_max_*</code>	blank = ∞	Resistor tolerance bands, $R_{min} \leq R < R_{max}$. Typing a maximum opens the next band, which starts there.
tol %	<code>rtol_tol_*</code>	1.00	Tolerance of the resistors in that band.
✕ remove last band	<code>rtol_rm</code>	—	Removes the last band.
Capacitor tol (%)	<code>resp_ctol</code>	5.00	Tolerance of every capacitor.

Control	Key	Default	What it does
Runs	<code>resp_runs</code>	2000	10–20000. Above 1000 a caption warns it may take a few seconds.
Distribution	<code>resp_dist</code>	Gaussian (tol = 3σ)	Gaussian: the tolerance is three standard deviations. Uniform: equally likely anywhere within $\pm\text{tol}$.
Seed	<code>resp_seed</code>	0	The same seed and inputs give the same band — reruns are reproducible.
Envelope	<code>resp_envelope</code>	p1–p99	Which percentiles the shaded band spans: p1–p99, p5–p95 or min–max.
Run Monte-Carlo	<code>resp_mc_run</code>	—	Runs it and draws the band; a caption gives the DC-gain spread against the nominal. After any change to the inputs or the picks, a caption asks you to run it again.

The same tolerances, distribution and runs go into the LTspice Monte-Carlo file (§9) and, when it is ticked, the report.

Bode — design vs realized

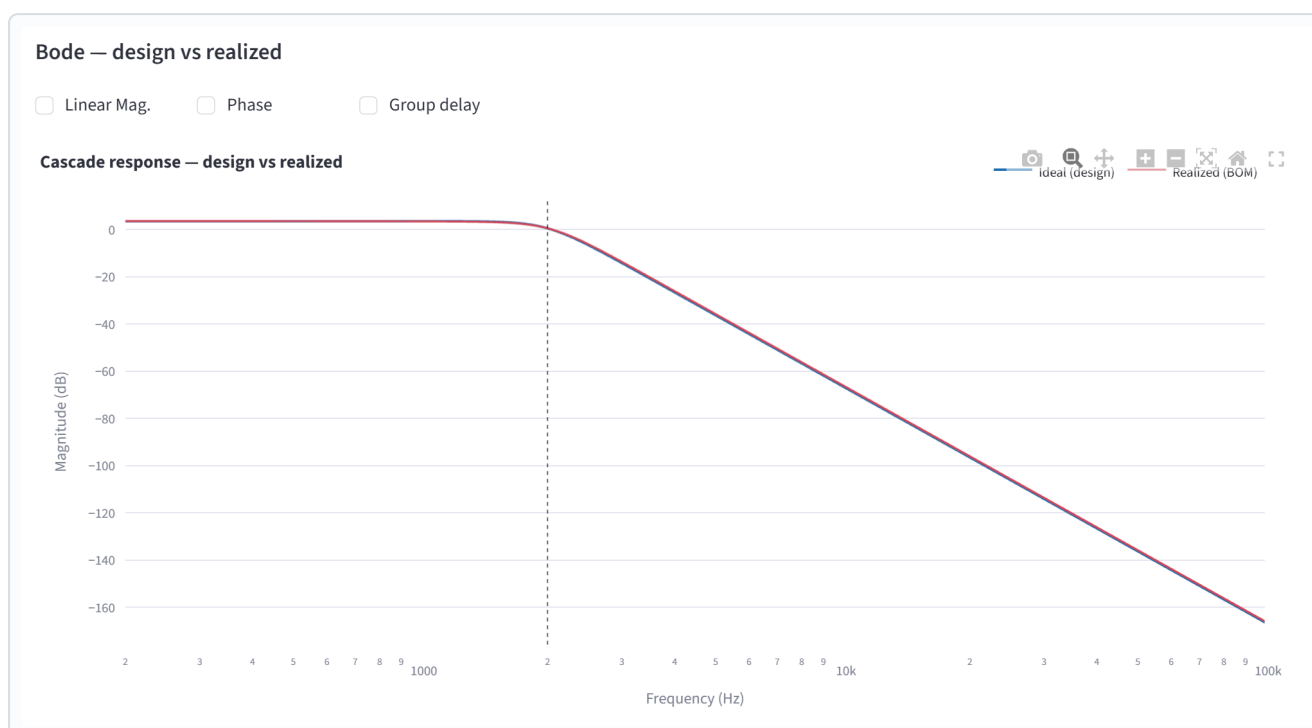


Figure 28. The realized cascade (red) on top of the design (blue).

Control	Key	Default	What it does
Linear Mag.	<code>resp_show_linear</code>	off	Linear magnitude axis.
Phase	<code>resp_show_phase</code>	off	Adds the phase.
Group delay	<code>resp_show_gd</code>	off	Adds the group delay, design and realized.

The plot draws Ideal (design) in blue, Realized (BOM) in red, the Monte-Carlo band in grey with its median dashed, and a vertical line at each section's f_0 (and f_z). The realized curve is the product of the sections computed one by one, each with its op-amp model; the loading of one section by the next is not included — the LTspice export simulates it (§9).

Section schematics, a collapsed expander below the plot, stacks every section's schematic in cascade order with its own **↓ SVG** / **↓ PNG** buttons — the same drawings as in Tab 3.

LTspice export

The block before the report writes the whole cascade as LTspice schematics and netlists, nominal and Monte-Carlo, in one zip. Chapter 9 covers it, with the vendor model import.

Report options

Generate Report

A4 PDF summary: specification, transfer function, pole-zero map with section pairing, per-section schematic + BOM, and the design-vs-realized Bode page.

☒ Cover page
 ☒ Polynomial coefficient table
 ☒ Normalized prototype roots

☒ Per-section quality metrics ⓘ
 ☐ Design warnings ⓘ
 ☒ Phase & group delay on Bode page

☒ Monte-Carlo band + parameters
 ☒ Passband detail plot ⓘ

Report title

Filter Synthesis Report

Project

Author

Generate Report

Figure 29. The Generate Report block. Everything is on by default; Design warnings is greyed when there are none.

Control	Key	Default	What it does
Cover page	rep2_cover	on	A title page. The three boxes below appear only while it is ticked; unticked, the report uses the default title.
Report title	rep2_title	Filter Synthesis Report	Heading of page 1 and every footer.
Project	rep2_project	blank	Big title on the cover (blank: <i>Active Filter Design</i>).
Author	rep2_author	blank	On the cover.
Polynomial coefficient table	rep2_coeff	on	Report §2a/2b.
Normalized prototype roots	rep2_norm	on	Report §4a.
Per-section quality metrics	rep2_metrics	on	Sensitivity, snap cost, realized against target gain, per section.
Design warnings	rep2_warn	on	Carries the HF-resonance notes (§5) into the report, §8. Greyed when none were raised.
Phase & group delay on Bode page	rep2_phgd	on	Adds them to report §6.
Monte-Carlo band + parameters	rep2_mc	on	The band, its settings and the passband-gain spread. If Monte-Carlo has not run with the current inputs, generating runs it once — a caption says so.

Control	Key	Default	What it does
Passband detail plot	rep2_zoom	on	Report §7: the band of interest only. Stopband detail for a band-reject.
 Generate Report	report_btn	—	Builds the PDF. Greyed variants: report_btn_blocked (a section has no pick), report_btn_nodata and report_btn_nobackend (§6.5).
↓ Download PDF	report_dl	—	Saves <i>FilterReport_<Response><Type>_n<order><date>_<time>.pdf</i> — <i>FilterReport_Custom_<Type>n<poles>...</i> for Custom H(s). Change anything afterwards and a caption says the PDF is out of date.

4. Choosing a topology family

Each section chooses its own family, so one cascade can mix them.

	VCVS (Sallen-Key)	MFB (Friend)	AM (Ackerberg–Mossberg)
Op-amps per section	1	1	3
Polarity	Non-inverting; the pure notch inverts	Inverting; the notch cells do not	Inverting
Low-pass, high-pass, with or without notch	Yes	Yes	Yes
2nd-order band-pass, pure notch	Yes	Yes	Yes
3rd-order band-pass (absorbed real pole)	No	Yes — the default for these sections	No
Near-notch section (zero just off f_0)	Needs a spread of $\sim 1/ (f_z/f_0)^2 - 1 $	Same as VCVS	No such limit
Section gain below 1	2nd-order low-pass; any high-pass	Any section	Any section
Strength	Fewest parts	Covers every section kind; gain is a free ratio	Q barely moves with op-amp bandwidth
Cost	Q sensitivity grows with Q	Inverts; Q-enhanced twins trade sensitivity for spread	Three op-amps; needs a low-output-resistance op-amp

MFB solves each cell together with its Q-enhanced ($-QE$) positive-feedback twin and ranks them; the plain cell usually wins where Q is modest. **AM** gets its accuracy from the matched pair $R7 = R8$, which compensates the op-amp's finite bandwidth: its Q error grows with $(f_0/f_t)^2$ instead of $Q \cdot f_0/f_t$.

Rules of thumb:

- **Start with VCVS.** It is the cheapest realization of low-pass and high-pass sections.
- **A 3rd-order band-pass section needs MFB.** The app starts such a section on MFB; if you prefer VCVS or AM, untick absorption in Tab 2 so the real pole becomes its own 1st-order section.
- **A near-notch section that returns nothing wants AM.** A zero 1–5 % from f_0 costs VCVS and MFB a large component spread (§3.4).
- **A section with Q below 0.5** comes from two real poles paired into one 2nd-order section (a wide band-pass or band-reject produces them). It is realized like any other 2nd-order section; VCVS handles it well.
- **A section gain below 1** on a 3rd-order low-pass needs MFB or AM, or move that gain to another section in Tab 2.
- **A real op-amp drags the realized curve away near a high-Q f_0 :** try AM with a low- R_o op-amp.
- **Polarity matters:** count the inverting sections; the Overall filter readout warns when the total is odd.
- **No BOM at all:** before changing family, read the diagnosis (§6.4) — it often says the family is fine and the envelope is not.

5. Op-amps and what they do to the design

The op-amp is chosen per section, because a high-Q or high-gain section demands more bandwidth than the others — or once for all sections in Batch mode (§3.4).

The shipped library, in the order of the list. The values are typical datasheet figures; check the datasheet for your supply and load, and adjust a part with  **Edit** (§3.4) — your edit is stored as an override.

Library entry	A _{ol} (V/V)	GBWP	R _o
Ideal (no op-amp limits)	∞	∞	0
AD8505 / AD8506 / AD8508	$1 \cdot 10^5$	95 kHz	1 k Ω
LMV358A	$1 \cdot 10^5$	1 MHz	1.2 k Ω
MAX9636 / MAX9637 / MAX9638	$1 \cdot 10^5$	1.5 MHz	100 Ω
MAX40100	$1.41 \cdot 10^6$	1.5 MHz	100 Ω
TL072H	$5.6 \cdot 10^5$	5.25 MHz	125 Ω
LM358B	$1.4 \cdot 10^5$	1.2 MHz	300 Ω
OPA1656	$5 \cdot 10^6$	20 MHz	26 Ω
TLV9001 / TLV9002 / TLV9004	$1 \cdot 10^5$	1 MHz	1.2 k Ω
Custom...	your values	your values	your values

Your own parts follow the shipped ones in the list. Each part also names the SPICE model the LTspice export uses for it (§9).

- **A_{ol}** — finite DC gain: small gain and Q errors.
- **GBWP** — the op-amp's gain falls with frequency. This shifts f_0 and raises Q, more so for high-Q, high-gain sections and as f_0 approaches the GBWP.
- **R_o** — output resistance works against the feedback network and shows up far above the passband, as a rise of the realized response.

With a real op-amp the solver pre-distorts the ideal solution so that the section lands on target *with* that op-amp, then snaps and checks the BOM against the exact non-ideal model. Tab 4's realized curve and the report use the same model; Monte-Carlo holds it fixed. The part number (or the Custom parameters) is printed under the op-amp in the schematic.

The HF resonance warning. Tab 4 compares realized with design from the highest corner up to 100 times it. When red rises at least 1 dB above blue, a yellow box appears:

⚠ HF resonance (~N dB at F) — over [range] the realized (red) response peaks ~N dB above the design (blue). This is the Ackerberg–Mossberg finite-R_o loop resonance from the op-amp output impedance (AM section ...), not a synthesis error.

Without AM sections it reads **⚠ HF rise** and blames the output impedance alone. It is physics, not a solver fault. Fix it with an op-amp of lower output resistance (about 50 Ω or less), or with MFB for the sections it names.

6. When it goes wrong

Messages are quoted as the app prints them; "..." marks the parts that vary.

6.1 Specification and approximation

The app says	What it means	What to do
Mathematical Constraint Violation: Band-Reject filters require an EVEN total order. Your current total is N.	Asymmetric band-reject with LP + HP odd.	Change one of the two orders by one.
Mathematical Constraint Violation: Asymmetric Elliptic Band-Reject requires ...	Asymmetric elliptic band-reject needs even orders that differ by at most 2.	Adjust the orders.
⚠ Max total order for Asymmetric Elliptic is 15. Extreme orders may cause calculation delays.	A warning, not a block.	Lower LP + HP, or be patient.
Engine Error: ...	The approximation engine failed for this specification.	Change the specification slightly (order, corners, A _s). If it persists, report it (§6.7).
Stopband Attenuation does not meet the requirements (Asymptote exceeds limit)	The stopband never reaches A _s — typically after pinning notches.	Unpin or move notches, raise the order, or lower A _s .
Calculated Stopband Edge (f _s): ... (Extended due to manual notch placement)	A pinned notch pushed the stopband edge out.	Informational. Move the notch if f _s matters.
Order is too low to support finite transmission zeros.	No notch rows at this order.	—
Manual Notch Tuning ... is disabled ...	Elliptic band filters, and Elliptic above order 8.	By design.

Bessel and Equiripple Delay

The app says	What it means	What to do
n = N — no order ≤ M meets the spec; this is the closest (yellow box, X lines)	No allowed order meets the order criterion; the design uses the closest one.	Relax the criterion — a longer τ ₀ budget, a lower f _d , a looser ε or δ, a lower A _s or a higher f _s — or accept the closest order.
Tick at least one order criterion. Using n = N meanwhile. / Order selection failed: ... Using n = N meanwhile.	No order could be selected; the design keeps the last manual order.	Fill in the criterion's value; report a failure (§6.7).
Stopband criterion ignored: f _s must lie outside f ₁ ...f ₂ .	Bandpass: the stopband frequency is inside the passband.	Move f _s below f ₁ or above f ₂ .
Band too wide for the delay-preserving bandpass: B/f ₀ = ... exceeds ... for n = N; the translated poles would cross the real axis.	The delay-preserving mapping works up to a fractional bandwidth that falls with the order.	Narrow the band, lower the order, or choose Classic (geometric) .

The app says	What it means	What to do
Equiripple Magnitude Stopband: needs manual order and the corner anchor.	The equiripple stopband places notches around a fixed corner and order.	Order selection Manual and Specify by Corner frequency .
Equiripple stopband: the notch solve did not converge (humps within ... dB of $-A_s$).	The notches were placed, but not all humps reached $-A_s$ exactly (the readout turns yellow).	Use fewer active notches, or a lower A_s .
Notches at or inside the passband were ignored: they cannot keep the delay flat (a passband notch adds a delay bump of area π).	A pinned notch below the corner.	Pin it above the corner.
The corner cannot be held at f_c : the notches are too close to it (...). Poles left unscaled; the corner moves down.	The notches pull the response down at the corner more than α allows.	Move the notches away from the corner.
Manual notches are not available for ... bandpass filters.	By design.	—

Custom H(s)

A refused H(s) shows a red box under the panel — **This H(s) cannot be used:** — followed by one line per reason. The design stops there; fix the entry and it continues.

The line says	What it means	What to do
'...' is not a number. / Coefficients must be finite. / ... must be a nonzero number.	A cell or constant cannot be read.	Correct the cell. Typed text is kept as typed, so a stray character shows.
The numerator / denominator is empty or all zero. / The denominator has no s terms: no poles. / H(s) has no poles.	Not a filter yet.	Fill in the denominator.
Pole pair N: f_0 must be > 0 . / Pole pair N: Q is needed ... / Row N: a and b are both 0. / Row N: a = 0 puts the poles on the $j\omega$ axis.	A row of a factored form is incomplete or puts poles on the $j\omega$ axis.	Complete or correct the row.
...: lone complex root ... (no conjugate).	A complex root was entered without its conjugate.	In the Roots form enter $\text{Im} \geq 0$ only — $\omega > 0$ stands for the pair.
N pole(s) at the origin / on the $j\omega$ axis ($Q = \infty$) / in the right half-plane ...	The filter would not be stable.	Check signs: poles need a negative real part.
Real LHP zero(s) at ... / Off-axis LHP zero pair(s) at ... / Real RHP zero(s) ... / RHP zero pair(s) ...	Only zeros at the origin, on the $j\omega$ axis or at infinity are realizable today; no cell has a real or off-axis zero.	Remove those zeros. An all-pass or equalizer H(s) cannot be built yet.
Improper H(s): N finite zeros $>$ M poles ... / Improper prototype ...	More zeros than poles.	Check the numerator degree.
N poles exceed the limit of 30. / Prototype order N exceeds the limit of M for ...	Above the order limits (§3.1).	Lower the order.
N $j\omega$ zero pairs but only M complex pole pairs ...	Each notch needs a complex pole pair to sit on.	Check the zeros, or the poles.

The line says	What it means	What to do
A lowpass prototype has no zeros at the origin.	Prototype mode with an origin zero.	Use Complete H(s) for a high-pass or band-pass you already have.
$ H(j1) $ is ... dB below the prototype's peak: $\omega = 1$ must be its passband edge (check the table's normalization).	Prototype mode: the entered prototype's edge is not at $\omega = 1$ (more than 40 dB down there).	Rescale the prototype, or use Complete H(s) with the table's f_n .
The prototype needs a corner frequency > 0 . / The prototype needs $0 < f_1 < f_2$.	Sidebar corners.	Correct them.
The response is not recognised as a lowpass, highpass, bandpass or band-reject at $\alpha = \dots$ dB ... Change α or H(s).	Complete mode: the type could not be detected at that edge level.	Change Passband edge level α , or check H(s).

Yellow warnings under the diagnostics do not stop the design:

The app says	What it means
$K < 0$ is used as $ K $...	The sign is set by the circuits, not by H(s).
N cluster(s) of repeated roots merged at the typed precision ...	Nearly equal roots were treated as one repeated root; enter them factored or as roots for exact multiplicity.
Coefficient conditioning: the typed precision leaves ... dB of passband uncertainty ...	A high-order polynomial typed with few digits; enter it factored or as roots.
$ H(j1) $ is ... dB below the peak: $\omega = 1$ is not the prototype's passband edge ... / The prototype does not look like a lowpass (detected: ...).	Prototype mode: check the table's normalization.
Detected type: ..., but the Filter Type radio says ...	Prototype mode: the mapped result is not what the radio says; the radio's type is used.
Entered peak gain $G = \dots V/V < 1$: an attenuating passband ...	Some VCVS sections may be unsolvable (a 3rd-order VCVS low-pass cannot go below 1); MFB and AM still work.
Pairing pre-flight: ... stage N (order ...) gets ... / has a numerator no cell realizes ...	The automatic pairing would give a section something no circuit realizes. Re-pair the stages named by hand in Tab 2.

6.2 Pairing

The app says	What it means	What to do
 Pairing Incomplete: There are floating zeros on the board (drawn in gray). Please assign them to a stage.	A manual edit left zeros without a section; nothing is handed to the Topology tab.	Click a pole, then the grey zero — or press  Auto-Pair (Reset).
 Please click a Pole first to initiate routing.	A zero was clicked first.	Click a pole, then the zero.
 Cannot merge poles: 'Enable 3rd-Order Sections' is unchecked.	Absorbing a real pole needs the checkbox.	Tick it.
 Manual Override Active: Background auto-router is currently bypassed.	Your edits are in charge.	 Auto-Pair (Reset) to go back.

6.3 Topology, before and during a solve

The app says	What it means	What to do
Finalize the cascade in Biquad Pairing & Cascading first — each section will appear here for hardware synthesis.	No cascade yet.	Complete the pairing in Tab 2 (§6.2), or fix the specification.
 ... section (family ...) — solver not yet available; gated to avoid mis-solving on a mismatched cell.	No solver exists for this kind of section yet.	Re-pair so the section becomes a supported kind — e.g. untick absorption.
Section N is a 3rd-order band-pass ... Switch this section to MFB (Friend) ...	VCVS and AM cannot place the third pole of a band-pass. Solve is greyed. Such a section starts on MFB; the message appears when you switch it away.	Family → MFB, or give the real pole its own section.
No R series selected — defaulting to E48.	No resistor series ticked.	Tick at least one.
 E3 must be added to <code>unified_solver_v2._E_SERIES</code> , else the cap grid is empty.	E3 capacitors are not enabled in the solver yet.	Use E6 or E12.
 Solving... (other sections and tabs stay responsive)	Normal.	Keep working elsewhere.
 Queued — waiting for a free core (other sections and tabs stay responsive)	More sections are solving than there are free cores (§1).	Wait; it starts when a core is free.
 Near-notch section: f_z is ...% from f_0 ... Solved on both ... and the pure-notch ...; the BOMs are ranked together by snap cost.	Informational (§3.4).	—
 The ... solve failed: ...	A near-notch section: one of its two solves failed; the table shows the other's BOMs.	Usable as is; report the failure (§6.7).
Op-amp library: ... could not be read (...); ignored. / ... duplicate key ... / ... entry skipped — ... / '...' is a reserved name; entry skipped.	A library file — usually your own, edited by hand — has an error. The yellow box above the picker names it; the rest of the library works.	Fix or delete the entry in the file the message names (§7.4).
Enter a name. / '...' conflicts with existing part '...' / '...' is reserved. / Name is longer than ... characters.	Saving an op-amp part: the name is empty, taken (case does not count), or reserved (<i>Ideal</i> , <i>Custom</i>).	Choose another name.
Could not save: ... / Could not update the library: ...	The user library file cannot be written.	Check that <code>%LOCALAPPDATA%\FilterSynthesizer</code> is writable.

6.4 A solve that returns nothing

When a solve finds no BOM, the app runs a quick idealized check of what the section would need and replaces the generic message with a diagnosis. The diagnosis can take up to half a minute after the solve ends.

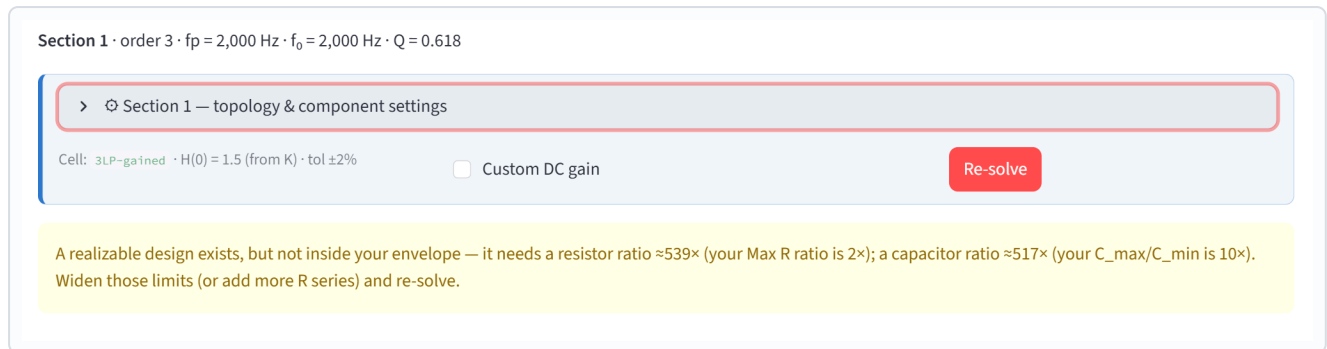


Figure 30. The envelope is too tight: the message names the resistor and capacitor ratios the design needs, and the limits you set.

The app says	What it means	What to do
No realization inside the component envelope. Widen R/C limits, relax Max R ratio, add more R series, or raise thoroughness.	Nothing found, and the check reached no verdict.	In that order.
A realizable design exists, but not inside your envelope — it needs a resistor ratio $\approx N\times$ (your Max R ratio is $M\times$); a capacitor ratio $\approx \dots\times$ (your $C_{\max}/C_{\min}$ is $\dots\times$). ...	The design needs a wider spread than you allow; each part names the limit that stops it.	Raise Max R ratio above N ; widen $C_{\min}\dots C_{\max}$ beyond the capacitor ratio named.
A realizable design exists inside your envelope (it needs only ...), but the search didn't reach it. ...	The limits are fine; the search missed it.	Search thoroughness → Thorough and re-solve, or nudge the section gain slightly.
A realizable design almost certainly exists at this gain (...), but the search didn't find one in your envelope. ...	The same, found by a gain-band check.	The same.
No realizable solution at this gain. You asked for X , but the nearest gain this topology can realize for the target shape is $\approx Y$	This family cannot make this gain with this f_0 and Q (and f_z).	Set the section gain to about Y with Custom gain and move the difference to another section, or change the family.
No realizable solution exists for these targets — the pole/zero shape itself isn't achievable with this topology. Choose another topology.	Structural.	Change the family (§4).
Near-notch section: the zero is ...% from f_0 , and VCVS / MFB notch cells need a component spread of about $1/ (f_z/f_0)^2 - 1 \approx N\times$ to place it. The AM (Ackerberg–Mossberg) family realizes it without that spread.	Shown above the diagnosis for a near-notch section on VCVS or MFB.	Family → AM, or raise Max R ratio and widen $C_{\min}\dots C_{\max}$ beyond $N\times$.
No realization fits the constraints — raise $C_{\max}$, add a resistor E-series, or (ni-gained) relax the gain so $R3+R4$ lands in 5k–50k.	A 1st-order section found no parts.	As it says.

6.5 Results and the report

Resulting Response & Schematic

Pick a BOM for every section in the **Topology** tab to see the cascade response (still pending: section 1, 2).

Generate Report

⚠ Report generation is not available yet — section 1, 2 still have no selected BOM. Solve and pick a row for every section in the **Topology** tab.

 Generate Report

Figure 31. Solved is not picked: the report stays blocked and names the sections without a BOM.

The app says	What it means	What to do
Select a solution for every section to see the overall gain (still pending: section N).	Tab 3's Overall filter waits for picks.	Tick a row in each section listed.
⚠ Output is inverted — an odd number of inverting (inv) stages gives overall sign -1. ...	The cascade inverts.	Make the count of inverting sections even (family or Realization), or add an inverter.
Finalize the cascade and synthesize sections in the Topology tab first — this view combines the selected per-section BOMs.	No cascade yet.	Tabs 3 and 4 first.
Pick a BOM for every section in the Topology tab to see the cascade response (still pending: section N).	Solved is not picked.	Tick a row in each section listed.
⚠ Report generation is not available yet — section N still has no selected BOM. ...	The same gate, for the report.	The same.
Section N: can't build response for cell (...).	The response model of that cell failed.	Pick another row; report it (\$6.7).
⚠ HF resonance (~N dB at F) ... / ⚠ HF rise (~N dB at F) ...	Op-amp output resistance (\$5).	A lower- R_o op-amp, or MFB for the AM sections named.
Inputs changed since the last run — click Run Monte-Carlo to refresh the band.	The band on screen is stale.	Run Monte-Carlo again.
Monte-Carlo failed: ...		Report it.
Monte-Carlo has not been run (or its inputs changed). Generating the report will run it once with the settings above — this adds a few seconds.	Informational.	—

The app says	What it means	What to do
Settings changed since this PDF was built — press Generate Report again to refresh it.	The download button still offers the older PDF.	Generate again.
Report generation failed: ...		Report it.
PDF report support needs the <code>reportlab</code> package, which is not installed in this environment.	Optional packages missing (source install).	<code>pip install reportlab matplotlib cairosvg</code>
Report data is not being published by <code>app.py</code> — missing ...	The installation is incomplete or modified.	Reinstall.
⚠ Schematic SVG not found ... expected ... in ...	The drawing for that cell is missing.	Restore the <code>Section_Schematic_Diagrams</code> folder (§7.4).

6.6 Errors with a Details box

Engine Error: ... (under the title) and **Solver error:** ... (in a section) each come with a collapsed **Details (paste this into a bug report)** box holding the technical trace. The first is the approximation engine, the second the hardware solver. Neither is your fault; report both (§6.7).

6.7 Reporting a problem

Include:

- the app version from the title line (*Filter Synthesizer v1.1.0 - ...*);
- the specification — the sidebar, or page 2 of a PDF report;
- the section's header line (order, fp, f₀, Q, f_z) and a screenshot of its open settings expander;
- the text of the **Details** box, when there is one;
- for an LTspice problem, the README.txt of the export zip (it lists every section's cell, model and warnings) — but not the vendor model files;
- for the packaged app, the output of `FilterSynthesizer.exe --selftest`, which writes a diagnostic report without starting the app.

A maintainer may ask you to start the app with the environment variable `FILTERSYNTHESIZER_DEBUG=1`; each section then shows a  **exact solver call** panel whose contents belong in the report.

6.8 No ↓ PNG button next to ↓ SVG

The PNG button appears only when the `cairosvg` package can render — it is a live indicator. Without it you lose the PNG downloads, and the report's schematic pages print a placeholder; everything else works. From source: `pip install cairosvg` (on Windows it also needs the Cairo library). In the packaged app, use the SVG.

6.9 LTspice export

The app says	What it means	What to do
... (section N): its vendor model is not imported yet, so it uses FS generic with the library's A_ol / GBWP / Ro. Download the model from ... and import it under Vendor model files.	The part needs a vendor model you have not imported (§9.3). The zip still works, with the generic model.	Import the model, or leave it if the generic model is enough.
Section N: SPICE model '...' is not in the op-amp model library / fails the dummy check (...) -- FS generic used	The part's SPICE model field names a model the export cannot use.	Correct the field in  Edit (§3.4), or clear it to use the generic model on purpose.
Section N: Vs = ... V is outside ...'s supply range ... V	The supply does not suit the real model.	Change Supply Vs .
Section N (...): no DC path at ... -- a real op-amp model will not bias there	That node floats at DC (the DC path column says the same). Harmless with the generic model.	With a real model, expect LTspice to fail the operating point there; pick another row or family.
Schematic (.asc) not written -- the drawing failed its self-check ...	The .asc schematics were left out so that no wrong drawing ships; the netlists (.cir) are complete.	Simulate the .cir files; report it (§6.7).
Section N: template ... rejected: ...	A cell template could not be used; the section is auto-laid-out instead.	Nothing — it simulates the same.
Op-amp library: ...	An op-amp model file in your library has an error (the Op-amp model library expander shows which).	Fix or remove the file named.
LTspice export failed: ...	The export could not be built; the rest of the tab is unaffected.	Report it (§6.7).

Importing a vendor model (§9.3):

The app says	What it means	What to do
...: not a readable zip file / the zip holds no ... / expected one of ...	The file chosen is not the vendor's model or zip for this part.	Download the PSpice model from the product page again; pick the zip itself.
...: file larger than ... MB	Not a model file.	Pick the model file or its zip.
no SPICE .subckt model found	The file holds no subcircuit.	Pick the model file (.lib, .txt, .mod, .cir ...), not a document.
Pin roles could not be read from the model — set each from the vendor's pinout note ...	The pin names did not say which pin is which.	Set each pin from the comment above the model's <code>.subckt</code> line.
give each of the 5 pins a different role (in+, in-, V+, V-, out)	Two pins have the same role.	Correct the pin roles.

The app says	What it means	What to do
installed — not bundled (no consent recorded)	A model file placed in the models folder by hand.	Press I accept the vendor's terms for ... to have it bundled.
Could not record the consent: ...	The models folder is not writable.	Check %LOCALAPPDATA%\FilterSynthesizer\LTspice_Library .

7. Reference tables

7.1 E-series

Series	Values per decade	Usual tolerance class	Offered for
E3	3	wider than $\pm 20\%$	capacitors (not usable yet)
E6	6	$\pm 20\%$	capacitors
E12	12	$\pm 10\%$	capacitors, resistors
E24	24	$\pm 5\%$	resistors
E48	48	$\pm 2\%$	resistors
E96	96	$\pm 1\%$	resistors

The series decides which values the solver may use. The tolerance of the parts you actually buy goes into Tab 4's Monte-Carlo.

7.2 Defaults

Setting	Default	In real parts
C_min ... C_max	6.80e-05 ... 1.00e-02 μF	68 pF ... 10 nF
R_min ... R_max	0.3 ... 2000 k Ω	300 Ω ... 2 M Ω
Max R ratio	500	
Capacitor / resistor series	E12 / E48	
Family / op-amp	VCVS / Ideal (MFB for a 3rd-order band-pass section)	
Batch mode	off	
Monte-Carlo	R 1 %, C 5 %, Gaussian, 2000 runs, seed 0, p1-p99	
LTspice export	Vs 5 V (± 2.5 V), MC runs = Monte-Carlo runs, vendor models used where imported	

7.3 Reading a cell name

Part	Means	Examples
Leading digit	Section order	2LP-unity , 3LP-gained
LP, HP, BP, N	Low-pass, high-pass, band-pass, pure notch	2HP-gained , 2BP , 2N
n after LP or HP	With a notch (a zero pair)	2LPn-unity , 3HPn-MFB

Part	Means	Examples
2BP1LP, 2BP1HP	Band-pass pair plus an absorbed low- or high-pass pole: a 3rd-order band-pass	2BP1HP-MFB
-unity, -gained, -atten	VCVS gain variant: = 1, > 1, < 1	2LP-atten
-MFB, -AM	Family; no suffix means VCVS	2LP-MFB, 3LP-AM
-MFB2, -AM2	The family's second realization: MFB2 the unity/gained HP-notch, AM2 the alternative output tap	2HPn-MFB2, 2LP-AM2
-QE	Q-enhanced twin (positive feedback)	2BP-MFB-QE
-LS	Lower-sensitivity branch	2LPn-MFB-LS
+R1, +R7, +R8	Twin with one extra resistor	2LPn-MFB-LS+R7
-C1s	Input capacitor split into a parallel pair	2HP-AM-C1s
1LP-ni, 1LP-inv	1st order, non-inverting / inverting, then the gain variant	1LP-ni-gained

7.4 Files and folders

Where	What
The folder holding FilterSynthesizer.exe	<code>_internal</code> (must stay next to the EXE); <code>Section_Schematic_Diagrams</code> — the schematic drawings, one <code>.drawio.svg</code> per cell, used without a rebuild when one is dropped there; <code>opamp_library.json</code> — the shipped op-amp library (§5); <code>LTspice_Library</code> — the shipped LTspice op-amp models, cell templates and symbols (§9); and the two PDF guides. Running from source, the same files sit in the app folder.
<code>%LOCALAPPDATA%\FilterSynthesizer</code> (Windows); <code>\$XDG_DATA_HOME/FilterSynthesizer</code> or <code>~/.local/share/FilterSynthesizer</code> (Linux)	The symbolic transfer-function cache and logs — safe to delete; the next start is slow again (10–20 s instead of 2–4 s). Running from source, the cache sits in the app folder. Your own files, kept across upgrades: <code>opamp_library_user.json</code> , your op-amp parts and edits (§3.4); <code>LTspice_Library\models</code> , the vendor models you imported, their <code>FS_<PART>.lib</code> wrappers and <code>consent.json</code> , the record of your consent (§9.3); <code>LTspice_Library\opamps</code> and <code>\cells</code> , your own op-amp models and cell templates, which override shipped ones of the same name. Delete these only if you mean to lose them.
Your browser's download folder	Reports, LTspice zips, SVG and PNG schematics.

7.5 Optional packages

Package	Without it
reportlab, matplotlib	No PDF report. The report block names the missing package and the pip line.
cairosvg	No ↓ PNG schematics; the report's schematic pages print a placeholder.

All three are in `requirements.txt`; the usual casualty is cairosvg, which needs the Cairo library on Windows.

8. What the PDF report contains

A4, the plot pages in landscape. Every page carries a footer: report title · date and time · design hash · page. The design hash identifies the cascade — two reports with the same hash describe the same sections and gains.

Part	Contents	Controlled by
Cover	Project (or <i>Active Filter Design</i>), the subtitle <i>Response Type · order N · corner</i> , author, date, app version	Cover page
1 · Design specification	Every sidebar value, and the calculated stopband edge(s). Bessel and Equiripple Delay add the delay specification — delay anchor, bandpass mapping, ripple δ , $\tau(0)$ or the delay at the centre, the flat-delay band, the peak-to-peak delay, the highest section Q, and how the order was chosen. Custom H(s) lists the mode, the entry form and scale, the structure (poles and zeros), the edges, the gain mode and the entered peak gain G instead of an order and corner; it has no modification or manual-notch rows. With the equiripple stopband, <i>Manual notches</i> lists the solved notches and the pole scale	always
2 · Transfer function	H(s), denormalized in rad/s, with the gain constant K; 2a monic coefficients; 2b coefficients with K folded into the numerator	2a/2b: Polynomial coefficient table
3 · Pole–zero map — section pairing	Each section's roots in its colour; dashed links pair poles with zeros; dotted circles mark each section's ω_0	always
4 · Root locations	Poles and zeros in rad/s with their section; 4a the normalized roots	4a: Normalized prototype roots
5.N · Section N — cell	Design parameters (order, f_0 , ω_0 , Q, f_{z_p} , f_{p_p} , K_i , Peak H); schematic; BOM with the pre-snap ideal in brackets; op-amp; solver constraints; quality metrics; the section's H(s)	Quality metrics: Per-section quality metrics
6 · Bode — design vs realized	Cascade magnitude with the Monte-Carlo band and its parameters; the passband-gain table	Phase & group delay on Bode page; Monte-Carlo band + parameters
7 · Passband detail	Design vs realized over the band of interest, same shading (<i>Stopband detail</i> for a band-reject)	Passband detail plot
8 · Design notes & warnings	The HF-resonance notes, when any were raised	Design warnings

Gain error in the quality metrics. With a single section it is measured against the specified filter gain. In a cascade each section carries only its share, so it is measured against that section's own design target. *DC gain (design)* is evaluated at the section's passband reference — f_0 for a band-pass, DC for a low-pass, HF for a high-pass; on a 3rd-order section the absorbed pole tilts the response, so it sits slightly below Peak |H|. *DC gain (realized)* uses the snapped values at zero tolerance.

Passband gain in §6 is the top of the passband ripple — the highest magnitude inside the band — found separately for the design, the realized curve and every Monte-Carlo run. It is not the DC gain: for an even-order Chebyshev or Elliptic without the even-order modification, DC sits a full ripple lower. This is also why

the report's spread can exceed the DC-gain spread shown in Tab 4. In the Quick Start example the screen gives 3.45 ... 3.71 dB at DC, yet with **Linear Mag.** ticked the band rises to about 1.6 V/V (+4.1 dB) just below the 2 kHz corner: tolerances raise a peak there, and the report's passband table, which takes each run's highest point, shows that wider spread.

9. LTspice export

The **LTspice export** block, in Tab 4 just above Generate Report, writes the whole cascade as an LTspice 24 simulation: the sections in series, exactly as picked, with each op-amp's SPICE model. It is the check the app cannot do itself — LTspice simulates the loading of each section by the next, and a real op-amp model adds what the app's linear model leaves out: rails, offsets and the part's own frequency behaviour.

LTspice export

LTspice schematics (.asc) and netlists (.cir) of the whole cascade — sections in series, so LTspice simulates the real inter-stage loading. Each op-amp uses its part's SPICE model from the local op-amp model library, or FS generic (the tool's own A_{ol} / GBWP / R_o). AC nominal and AC Monte-Carlo (tolerances from the settings above), plus a README with the expected probe values.

Supply Vs (V) ?

MC runs ?

5.00 - +

2000 - +

Section	Cell	Op-amp (tool)	SPICE model	Drawing	DC path
1	3LP-gained	Ideal (no op-amp limits)	FS generic · A_{ol} 1 G · GBWP 10 THz · R_o 0 Ω	template SK_LP	ok
2	2LP-unity	Ideal (no op-amp limits)	FS generic · A_{ol} 1 G · GBWP 10 THz · R_o 0 Ω	template SK_LP	ok

Inter-stage loading: the exported cascade differs from the tool's realized curve (a product of unloaded sections) by at most 0.000 dB within 60 dB of the peak. The README lists the expected LTspice probe values.

2000 Monte-Carlo runs: fast to simulate, slow for LTspice to plot.

> Op-amp model library

↓ Download LTspice files (.zip)

Figure 32. The LTspice export block: supply, Monte-Carlo runs, one row per section, and the download.

9.1 Controls and what the table says

Control	Key	Default	What it does
Supply Vs (V)	<code>spice_vs</code>	5	Written as two Vs/2 sources with ground at the midpoint, ± 2.5 V by default. The generic model is linear, so Vs matters only for real op-amp models; a Vs outside a part's supply range raises a warning.
MC runs	<code>spice_mc_runs</code>	the Runs above	Runs in the Monte-Carlo file, 1–20000. LTspice simulates thousands quickly but plots them slowly.
Export ... with simplified generic models	<code>spice_generic_vendor</code>	off	Shown when a section uses a part with a vendor model (§9.3). On: those sections use the generic model with the library's A_{ol} , GBWP and R_o . Off: the vendor model, where you have imported it.
↓ Download LTspice files (.zip)	<code>spice_dl</code>	—	The zip, <code>FS_LTspice_<design><date><time>.zip</code> . It is rebuilt on every change, so there is no separate Generate step.
Section N SPICE model	<code>spice_opamp_*</code>	Auto	Diagnostic only, shown when the app runs with <code>FILTERSYNTHESIZER_DEBUG=1</code> : overrides a section's model.

One row per section:

Column	Shows
Section, Cell	The section and its picked cell.
Op-amp (tool)	The op-amp chosen in Tab 3.
SPICE model	The model the export uses: the part's model, or <i>FS generic · A_{ol} ... · GBWP ... · Ro ...</i> — prefixed <i>Simplified</i> when the checkbox above is on, or <i>Model not imported</i> when the part's vendor model is missing.
Drawing	How the section is drawn in the .asc schematic: <i>template ...</i> (a hand-drawn cell template) or <i>auto-layout</i> (op-amps, a row of resistors and a row of capacitors, every pin on a labelled net) — with the reason when a template was rejected. Both simulate the same; the template is easier to read.
DC path	<i>ok</i> , or the nodes with no DC path to ground — a real op-amp model will not bias there.

Under the table, warnings — a model that is missing or failed its check (the generic model is used instead), Vs outside a part's range, a floating node, a vendor model not imported yet — and one caption:

- **all generic models:** how far the exported cascade can differ from the tool's realized curve because of inter-stage loading — at most so many dB within 60 dB of the peak. With Ideal op-amps it is 0.000 dB; a real op-amp's output resistance adds a little, mostly in the far stopband;
- **real models in use:** LTspice will differ from the tool by the models' own behaviour; run the commented `.op` once and check that every section output sits near 0 V.

Op-amp model library, a collapsed expander at the foot of the block, lists every op-amp model the export knows, where it comes from, whether it passed its check, and the folders: the built-in library, your own (§7.4), and the symbol calibration in use.

9.2 Which model each op-amp gets

The model follows the op-amp chosen in Tab 3 — its **SPICE model** field in the op-amp library (§3.4). There are three kinds:

Kind	Parts	What you need
FS generic	Ideal, Custom..., and any part without a SPICE model	Nothing. The app's own model, $A(s) = A_{ol} / (1 + s \cdot A_{ol} / (2\pi \cdot GBWP))$ behind R_o — linear, no rails, no offset, no noise, so LTspice reproduces the tool exactly. Ideal is written as $A_{ol} = 10^9$, $GBWP = 10$ THz, $R_o = 0$.
LTspice built-in	AD8505 / AD8506 / AD8508, MAX9636 / MAX9637 / MAX9638, MAX40100	Nothing: LTspice ships these models.
Vendor model	TL072H, LM358B, LMV358A, OPA1656, TLV9001 / TLV9002 / TLV9004	The model file from TI's product page, downloaded by you and imported once (§9.3). Until then the part is exported with FS generic and the library's values, with a warning.

9.3 Importing a vendor model

Vendor SPICE models are the vendors' copyrighted files. The app does not ship them and never downloads them for you; you download the model yourself and, by doing so, accept the vendor's terms. The import is done once per part and kept on this PC.

LTspice export

LTspice schematics (.asc) and netlists (.cir) of the whole cascade — sections in series, so LTspice simulates the real inter-stage loading. Each op-amp uses its part's SPICE model from the local op-amp model library, or FS generic (the tool's own A_{ol} / GBWP / Ro). AC nominal and AC Monte-Carlo (tolerances from the settings above), plus a README with the expected probe values.

Supply Vs (V)

MC runs

5.00

2000

☐ Export OPA1656 with simplified generic models

Vendor model files

Part	Sections	Model file	Status	Vendor page
OPA1656	1, 2	FS_OPA1656.lib	not imported — FS generic is used	https://www.ti.com/product/OPA1656

Your models folder: C:\Users\Manin\AppData\Local\FilterSynthesizer\LTspice_Library\models

Vendor SPICE models are the vendors' copyrighted files. FilterSynthesizer does not ship them and never downloads them for you. Download the model yourself from the vendor's product page (link below); by downloading it you accept the vendor's terms. A file you import here is stored only on this PC, in your models folder, and is copied unmodified into **your own** LTspice export zips, together with a small wrapper the tool generates around it, so they run as they are. Until a part's model is imported, its sections are exported with the FS generic model. Vendor terms typically allow use only for your own design work with the vendor's parts (TI: *only for development of an application that uses the TI products; other reproduction prohibited*) — **do not share export zips that contain vendor model files**; the README in each such zip says which files they are.

OPA1656 — import the vendor model

Open the OPA1656 product page ↗

☐ I download the OPA1656 model from the vendor myself, under the vendor's terms, for my own design work, and will not share export zips that contain it

The vendor's zip or model file, from wherever you saved it (.zip, .lib, .txt, .mod, .cir, ...)

Drag and drop file here
Limit 200MB per file

Browse files

Figure 33. Vendor model files: the status of each model, the vendor's page, the terms, and the import.

1. **Pick the part** in Tab 3 for at least one section and pick its BOM. The **Vendor model files** expander opens by itself while a model is missing.
2. **Open the product page** with the button (or the link in the status table) and download the part's PSpice model — usually a zip — to any folder.
3. **Tick the consent box:** *I download the ... model from the vendor myself, under the vendor's terms, for my own design work, and will not share export zips that contain it.* The file picker is greyed until you do.
4. **Choose the file** — the vendor's zip as downloaded, or the model file itself (.lib, .txt, .mod, .cir ...).
5. **Check the subcircuit and its pins.** The app finds the model's subcircuit and reads each pin's role (in+, in-, V+, V-, out) from the pin names or the vendor's pinout note. When it cannot, it says so — set each pin from the comment above the model's `.subckt` line.
6. **Import the ... model.** The status turns to *imported ...* — *bundled into the zip*, and the part's sections switch to the vendor model.

User Manual · v1.1.0

57

LTspice export

LTspice schematics (.asc) and netlists (.cir) of the whole cascade — sections in series, so LTspice simulates the real inter-stage loading. Each op-amp uses its part's SPICE model from the local op-amp model library, or FS generic (the tool's own A_{ol} / GBWP / Ro). AC nominal and AC Monte-Carlo (tolerances from the settings above), plus a README with the expected probe values.

Supply Vs (V)
MC runs

5.00
2000

☐ Export OPA1656 with simplified generic models

> Vendor model files

Section	Cell	Op-amp (tool)	SPICE model	Drawing	DC path
1	3LP-gained	OPA1656	OPA1656 · model FS_OPA1656	template SK_LP	ok
2	2LP-unity	OPA1656	OPA1656 · model FS_OPA1656	template SK_LP	ok

Real op-amp models in use: LTspice will differ from the tool's curve by the models' own behaviour. Run the commented `.op` once and check that every section output sits near 0 V.

This zip contains vendor model files — for your own use only; do not share it (see its README).

2000 Monte-Carlo runs: fast to simulate, slow for LTspice to plot.

> Op-amp model library

Download LTspice files (.zip)

Figure 34. After the import: both sections export with OPA1656's own model, and the captions say that real models are in use and that this zip contains vendor files.

OPA1656 — import the vendor model

Open the OPA1656 product page ↗

☒ I download the OPA1656 model from the vendor myself, under the vendor's terms, for my own design work, and will not share export zips that contain it

The vendor's zip or model file, from wherever you saved it (.zip, .lib, .txt, .mod, .cir, ...)

Drag and drop file here
Limit 200MB per file

Browse files

sbomaw6c.zip 27.9KB

Pin 1 **IN+**
in+

Pin 2 **IN-**
in-

Pin 3 **VCC**
V+

Pin 4 **VEE**
V-

Pin 5 **OUT**
out

Subckt OPA1656 in OPA1656.LIB . Pin roles read from the model's pin names / pinout note — check them.

Import the OPA1656 model

Figure 35. The import step: the model's subcircuit and its five pin roles, confirmed before the import.

Control	Key	What it does
Consent	<code>spice_vendor_consent_*</code>	Per part. Required before a file can be chosen.
The vendor's zip or model file	<code>spice_vendor_up_*</code>	The file to import, from wherever you saved it. A model already in your models folder is offered without a new download.
Model subcircuit	<code>spice_vendor_sub_*</code>	Shown when the file holds more than one subcircuit.
Pin <i>k</i>	<code>spice_vendor_role_**_*</code>	The role of each of the subcircuit's five pins; each role exactly once.

Control	Key	What it does
Import the ... model	<code>spice_vendor_inst_*</code>	Imports it. An error names what is wrong (a role used twice, an unreadable file).
Re-import the ... model (a new revision)	<code>spice_vendor_redo_*</code>	For a part already imported: shows the import again, for a newer model.
I accept the vendor's terms for ...: bundle them into my own zips	<code>spice_vendor_allow</code>	Only for model files placed in the models folder by hand, without consent recorded: records it, so they are bundled.

What is stored. The vendor's file, unchanged except for a few top-level directive lines that would break inside a cascade, as `<PART>__<file>`, and a small wrapper the app generates around it, `FS_<PART>.lib`, which maps the vendor's pins onto the app's op-amp symbol and keeps the vendor's helper subcircuits from colliding with another vendor's. Both go to your models folder — `%LOCALAPPDATA%\FilterSynthesizer\LTspice_Library\models` — together with a record of the consent. The status table shows that folder.

An export zip that contains vendor model files is for your own use. Vendor terms typically allow the model only for your own design work with the vendor's parts (TI: *only for development of an application that uses the TI products*). Do not share such a zip: the caption under the table says when a zip contains them, and its README names the files. To share a design, tick **Export ... with simplified generic models** first.

9.4 What is in the zip, and how to run it

File	Contents
<code><design>_AC.asc</code>	Schematic with nominal values: AC sweep over the tool's frequency grid, <code>.meas</code> probes, only V(OUT) saved so LTspice plots it at once.
<code><design>_AC_MC.asc</code>	The same with Monte-Carlo values: the tolerances, distribution and run count from Tab 4.
<code><design>_AC.cir</code> , <code><design>_AC_MC.cir</code>	The same two circuits as netlists; they open without any symbol or library.
<code>README.txt</code>	The design, the files, how to run, the circuit conventions, each section's cell, model and drawing, the Monte-Carlo settings, the expected probe values, warnings.
<code>FS_<PART>.lib</code> , <code><PART>__<file></code>	Only when a vendor model is used: the wrapper and the vendor's file. Keep them next to the <code>.asc</code> / <code>.cir</code> .

`<design>` is the specification in short, e.g. `Butterworth_Lowpass_n5`. Nominal and Monte-Carlo are separate files on purpose: LTspice's random functions never return the nominal value, and the nominal file keeps plain values you can read and edit. The schematics and netlists come from one model of the circuit and are checked against each other; if a schematic fails that check it is left out of the zip, with a warning, and the netlists remain.

Running it:

1. Extract the whole zip into one folder.
2. In LTspice 24, **File > Open** the `.asc` — or a `.cir`, with the file type set to *Netlists (.cir)**.

3. **Simulate > Run.** $V(\text{OUT})$ is plotted in magnitude and phase; right-click the phase axis and choose *Group Delay* for $\tau(f)$. Delete the `.save V(OUT)` line to probe internal nodes.
4. **View > SPICE Error Log** lists the `.meas` probes — one value per run in the Monte-Carlo file; right-click the log and choose *Plot .step'ed .meas data* to see their spread.

Checking the result. The README's *EXPECTED VALUES* gives, per probe frequency, the *loaded* value — the cascade exactly as written, which LTspice must match to about 0.01 dB with generic models — and the *tool* value, the app's realized curve. With real models, run the commented `.op` once first and check that every section output sits near 0 V; the expected values are then a passband sanity reference only.

Conventions in the circuit: input IN, output OUT, section k 's output S_k ; designators carry the section number — R201 is section 2's R1, a split capacitor is C202A / C202B, op-amp U201. The LTspice Monte-Carlo differs from the tool's in two small ways: LTspice draws its own random numbers, so only the statistics compare, and a split capacitor is two parts drawn independently (the tool varies their sum), which makes the tool's band slightly wider.